
ARTISYNC: Plataforma web de intermediación entre creadores de contenido digital y clientes con estrategia híbrida de acceso a datos, verificación asistida por IA y evaluación empírica multidimensional

Proyecto de Fin de Curso (PFC) — Aplicaciones Web, Quinto Nivel
Período Académico Presencial 2026-2027

Integrante	ORCID
Bone Arroyo, Niurca Scarleth	https://orcid.org/0009-0002-2219-2800
Carvajal Loor, Johan Stalin	https://orcid.org/0009-0008-9229-382X
Figueroa Morales, Bryan Javier	https://orcid.org/0009-0009-6357-4996
Rios Cuyabazo, Jhon Kevin	https://orcid.org/0009-0003-7446-9450

Docente-director: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.
gguerrero@uteq.edu.ec

Fecha de cierre: 17 de agosto de 2026 (semana 17)

Etiqueta del artefacto: v1.0.0 — commit d07656b

DOI del software (Zenodo): 10.5281/zenodo.21978572 (archivo de v1.0.0; la versión anterior, v0.9.0-rc, quedó archivada aparte en 10.5281/zenodo.21730559)

DOI del dataset de mediciones (Zenodo): Depósito separado, ver Bloque D.3 de la guía; brecha declarada en docs/observaciones/INFORME-BRECHAS-ENTREGA-FINAL.md

Repositorio: <https://github.com/JohanCarvajal04/Proyecto-WEB-ARTISYNC>

Resumen

Contexto y problema. La economía creativa digital carece de plataformas que centralicen, en un único entorno auditable, la contratación, la comunicación y el pago entre creadores de contenido y clientes, obligando a ambas partes a fragmentar el proceso entre redes sociales, mensajería externa y pasarelas de pago desconectadas — lo que eleva el riesgo de fraude, de incumplimiento contractual y de pérdida de trazabilidad financiera.

Objetivo. Diseñar, implementar y evaluar empíricamente Artisync, una plataforma web que integra en un solo sistema el ciclo completo de intermediación (perfil verificado, catálogo, mensajería moderada, contrato con firma electrónica, pago con patrón *escrow* y funciones sociales), aplicando una estrategia híbrida de acceso a datos (Object-Relational Mapping (ORM) para operaciones Create, Read, Update, Delete (CRUD) elementales, procedimientos almacenados PL/pgSQL para operaciones multi-tabla) y verificación de identidad asistida por inteligencia artificial. **Métodos.** Se siguió la metodología Design Science Research (DSR) de Peffers en seis actividades [1], con ingeniería de requisitos conforme a ISO/IEC/IEEE 29148:2018 [2] y al marco de Pohl [3], y evaluación empírica multidimensional: prueba de carga (k6, 50 VUs/30s), auditoría de seguridad (6 controles Open Web Application Security Project (OWASP) [4] + escaneo automatizado ZAP baseline + análisis estático SpotBugs/find-sec-bugs), usabilidad (System Usability Scale (SUS), $N = 16$) [5,6], cobertura de código (JaCoCo) y calidad web (Lighthouse, perfiles mobile y desktop). **Resultados principales.** El sistema alcanza p95 de 50.17 ms bajo carga (umbral 200 ms), 0 % de errores HTTP ≥ 500 , puntaje SUS de 76.88/100 (categoría B de Bangor, IC 95 % [69.16, 84.59]), cobertura JaCoCo de 72.0 % líneas / 62.5 % ramas, Lighthouse Performance 100/100 en desktop y 80–81/100 en mobile con Accessibility/Best Practices/SEO ≥ 93 en ambos perfiles, los 6 controles OWASP mínimos evidenciados sin hallazgos altos en ZAP baseline (0 FAIL-NEW), y 7 procedimientos almacenados conectados end-to-end al código en ejecución vía Jakarta Persistence API (JPA) (@Query(nativeQuery=true) parametrizado), sin concatenación de SQL dinámico detectada. **Conclusiones.** La estrategia híbrida de acceso a datos y la verificación asistida por IA son técnicamente viables dentro de las restricciones de un proyecto académico de 17 semanas, con evidencia empírica reproducible que respalda cada propiedad de calidad declarada; quedan como brechas honestamente declaradas la conexión completa de los procedimientos heredados de la Tercera Entrega, la ejecución de tests inferenciales sobre las comparaciones de rendimiento, y el despliegue en un ambiente de producción con dominio público.

Palabras clave: ingeniería de requisitos; procedimientos almacenados; verificación de

identidad; arquitectura de microservicios; usabilidad de software; seguridad web

Abstract

Context and problem. The digital creative economy lacks platforms that centralize, within a single auditable environment, the contracting, communication, and payment process between content creators and clients, forcing both parties to fragment the workflow across social media, external messaging, and disconnected payment gateways — raising the risk of fraud, contractual non-compliance, and loss of financial traceability. **Objective.** To design, implement, and empirically evaluate Artisync, a web platform that integrates the full intermediation cycle (verified profile, catalog, moderated messaging, electronically signed contract, escrow-pattern payment, and social features) into a single system, applying a hybrid data-access strategy (ORM for elementary CRUD operations, PL/pgSQL stored procedures for multi-table operations) and AI-assisted identity verification. **Methods.** The study followed Peffers’ Design Science Research methodology across six activities [1], with requirements engineering conforming to ISO/IEC/IEEE 29148:2018 [2] and Pohl’s framework [3], and multidimensional empirical evaluation: load testing (k6, 50 VUs/30s), security auditing (6 OWASP controls plus automated ZAP baseline scanning and SpotBugs/find-secc bugs static analysis), usability (System Usability Scale, $N = 16$), code coverage (JaCoCo), and web quality (Lighthouse, mobile and desktop profiles). **Main results.** The system achieves a p95 of 50.17 ms under load (200 ms threshold), 0 % of HTTP ≥ 500 errors, a SUS score of 76.88/100 (Bangor grade B, 95 % CI [69.16, 84.59]), JaCoCo coverage of 72.0 % lines / 62.5 % branches, Lighthouse Performance of 100/100 on desktop and 80–81/100 on mobile with Accessibility/Best Practices/SEO ≥ 93 on both profiles, all 6 minimum OWASP controls evidenced with no high findings in the ZAP baseline scan (0 FAIL-NEW), and 7 stored procedures connected end-to-end to the running code via JPA (parameterized `@Query(nativeQuery=true)`), with no dynamic SQL concatenation detected. **Conclusions.** The hybrid data-access strategy and AI-assisted verification are technically viable within the constraints of a 17-week academic project, backed by reproducible empirical evidence for every declared quality property; honestly declared gaps remain in fully connecting the stored procedures inherited from the third deliverable, running inferential tests on the performance comparisons, and deploying to a production environment with a public domain.

Keywords: requirements engineering; stored procedures; identity verification; micro-services architecture; software usability; web security

Índice general

Resumen	i
Abstract	iii
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Preguntas de investigación	2
1.3. Objetivo general y objetivos específicos	2
1.4. Contribuciones	3
1.5. Organización del documento	3
2. Marco teórico	5
2.1. Ingeniería de requisitos: ISO/IEC/IEEE 29148:2018 y el marco de Pohl	5
2.2. Calidad de requisitos: INCOSE Guide to Writing Requirements v4	5
2.3. Historias de usuario y casos de uso como técnicas complementarias	5
2.4. Modelo arquitectónico C4	6
2.5. Calidad de software: ISO/IEC 25010	6
2.6. Principios REST	6
2.7. Autenticación: JWT (RFC 7519)	6
2.8. Controles OWASP Top 10	7
2.9. Patrones de acceso a datos y modelos de consistencia	7
2.10. Documentación de decisiones de arquitectura (ADR)	7
3. Trabajos relacionados	9
3.1. Declaración honesta de alcance	9
3.2. Contexto mínimo defendible sin revisión sistemática	9
3.3. Brecha identificada (<i>research gap</i>)	11
4. Ingeniería de requisitos	12
4.1. Contexto y stakeholders	12
4.2. Técnicas de elicitación aplicadas	13
4.3. Requisitos funcionales y no funcionales: categorización y priorización	13
4.4. Modelado de casos de uso y de historias de usuario	14
4.5. Validación de requisitos	14
4.6. Gestión del cambio de requisitos	15

4.7. Métricas de calidad de requisitos	15
5. Materiales y métodos	17
5.1. Enfoque metodológico global	17
5.2. Instrumentos de medición	18
5.3. Enfoque de métricas: Goal-Question-Metric (GQM)	19
5.4. Población, muestreo y participantes	19
5.5. Protocolo experimental replicable	20
5.6. Análisis estadístico	20
5.7. Determinismo y semillas	21
6. Diseño del sistema y arquitectura	22
6.1. Modelo C4	22
6.1.1. Nivel 1 — Contexto	22
6.1.2. Nivel 2 — Contenedores	22
6.1.3. Nivel 3 — Componentes (backend)	24
6.2. Resumen de los siete registros de decisión de arquitectura (ADR)	24
6.3. Atributos de calidad (ISO/IEC 25010) — prioridad, escenario y estrategia	27
6.4. Modelo de datos	28
6.5. Matriz de trazabilidad	28
7. Implementación	30
7.1. Manejo de errores globales (RFC 7807 Problem Details)	30
7.2. Esquema de caching	30
7.3. Mecanismo de seguridad: JWT + rate limiting	31
7.4. Estrategia híbrida de acceso a datos: procedimiento almacenado representativo	32
7.4.1. Nuance de conformidad, declarada sin maquillaje	33
8. Evaluación empírica y resultados	34
8.1. Rendimiento (k6) — RQ1	34
8.2. Seguridad (OWASP + ZAP + análisis estático) — RQ1, RQ2	35
8.3. Usabilidad (SUS) — RQ1	36
8.4. Cobertura de código (JaCoCo) — RQ1	37
8.5. Calidad web (Lighthouse) — RQ1	38
8.6. Resumen de reproducibilidad de las figuras y tablas de este capítulo	39
9. Discusión	40
9.1. Respuesta a las preguntas de investigación	40
9.2. Comparación con trabajos relacionados	41
9.3. Hallazgos inesperados	41
9.4. Implicaciones prácticas	41

10.Amenazas a la validez	42
10.1. Validez de constructo	42
10.2. Validez interna	42
10.3. Validez externa	42
10.4. Validez de conclusión	43
11.Trabajo futuro	44
12.Conclusiones	45
13.Declaraciones obligatorias	46
13.1. Disponibilidad de código	46
13.2. Disponibilidad de datos	46
13.3. Disponibilidad de materiales	46
13.4. Declaración ética	47
13.5. Contribuciones de autores (taxonomía CRediT)	47
13.6. Declaración de conflictos de interés	47
13.7. Declaración de financiamiento	47
13.8. Declaración de uso de asistentes de inteligencia artificial	47
13.9. Agradecimientos	48
A. Tabla de observaciones acumuladas	53
B. Checklist Ralph et al. 2021	54
C. Checklist FAIR	55
D. Checklist INCOSE v4	56
E. Checklist PRISMA 2020	57
F. Matriz de trazabilidad completa	58
G. Protocolo SUS completo (10 preguntas + escala)	59
H. Capturas de ejecución del pipeline CI	60
I. Cadena de búsqueda completa del Capítulo 3	61
J. Clasificación de impacto de la bibliografía	62

Índice de figuras

6.1. Diagrama C4 Nivel 2 — Contenedores de Artisync. Fuente:	
docs/diagramas/c4-nivel2-contenedores.png.	23
6.2. Modelo entidad-relación de Artisync. Fuente: docs/diagramas/Entidad_-	
Relacion.png.	28
7.1. Diagrama de secuencia del flujo de autenticación JWT. Fuente:	
docs/diagramas/secuencia_login_jwt.png.	32
8.1. Diagrama de caja de los puntajes SUS ($N = 16$). Fuente:	
docs/mediciones/sus/boxplot-sus.png.	37

Índice de cuadros

4.1. Interesados por capa de cercanía al sistema	12
4.2. Distribución de requisitos por prioridad MoSCoW	13
4.3. Estado de validación de los 37 requisitos	15
5.1. Instanciación de las seis actividades DSR de Peffers et al.	17
5.2. Instrumentos de medición empírica utilizados	18
6.1. Resumen de los siete ADR de Artisync	25
6.2. Atributos de calidad ISO/IEC 25010 [7] priorizados	27
8.1. Resultados de la prueba de carga k6 por escenario	34
8.2. Controles OWASP evidenciados manualmente	35
8.3. Resultados del cuestionario SUS	36
8.4. Cobertura de código JaCoCo	38
8.5. Resultados Lighthouse por perfil y categoría	38
J.1. Clasificación de impacto por referencia	63

Lista de siglas y acrónimos

CRUD Create, Read, Update, Delete. I

DSR Design Science Research. I, 17

JPA Jakarta Persistence API. I

MoSCoW Must, Should, Could, Won't. 13

ORM Object-Relational Mapping. I

OWASP Open Web Application Security Project. I

SUS System Usability Scale. I

1. Introducción

1.1. Contexto y motivación

La economía creativa digital —el conjunto de actividades económicas en las que músicos, ilustradores, diseñadores y desarrolladores monetizan directamente su trabajo a través de internet— ha crecido de forma sostenida como alternativa de ingreso para profesionales independientes [8]. Sin embargo, la infraestructura que sostiene esa economía sigue fragmentada: un creador independiente típico coordina la comunicación con clientes por una red social o mensajería instantánea, gestiona el cobro por una pasarela de pagos genérica sin retención de garantía, y no cuenta con un mecanismo estandarizado de contrato ni de verificación de identidad de ninguna de las dos partes. Esta fragmentación introduce fricción operativa medible: la literatura sobre freelancers de plataformas digitales reporta la gestión de múltiples herramientas desconectadas y la ausencia de mecanismos de protección de pago como fuentes recurrentes de conflicto y abandono de la actividad [9]. El patrón de pago en garantía (*escrow*) —retener los fondos del cliente hasta la aprobación explícita del entregable— es la mitigación estándar de la industria para el riesgo de incumplimiento en transacciones entre partes que no se conocen previamente, formalizable incluso mediante protocolos de depósito dual verificables [10], pero su implementación correcta exige una arquitectura de datos con garantías transaccionales estrictas, que la mayoría de las herramientas genéricas de mensajería y pago no ofrece de forma nativa.

Artisync responde a esa fragmentación con una plataforma que centraliza, en un único sistema auditable, el ciclo completo de intermediación entre **Creadores** de contenido digital y **Clientes**: perfil verificado por un servicio de inteligencia artificial, catálogo dinámico de servicios y productos, mensajería moderada que bloquea el intercambio de datos de contacto externo (mitigación directa del riesgo de que las partes se muevan fuera del sistema auditable), contrato con firma electrónica generado desde plantilla, flujo de pedido por etapas con trazabilidad completa, pago mediante PayPal Orders v2 con patrón *escrow*, y funciones sociales (seguidores, comentarios, sorteos) para fomentar la retención de usuarios. El modelo de doble lado (Creador/Cliente) de Artisync es, en esencia, una plataforma multilateral en el sentido de Hagiu y Wright [11], orquestando transacciones entre dos grupos de usuarios que de otro modo dependerían de intermediarios genéricos.

1.2. Preguntas de investigación

RQ1. ¿Cumple el sistema, con evidencia empírica reproducible, los umbrales de rendimiento, seguridad, usabilidad, cobertura de código y calidad web declarados como requisitos no funcionales de la especificación (`docs/requisitos/SRS.md`)?

RQ2. ¿Es la estrategia híbrida de acceso a datos (operaciones CRUD elementales vía ORM, operaciones multi-tabla vía procedimientos almacenados PL/pgSQL invocados exclusivamente mediante los mecanismos formales de JPA 2.1) implementable y verificable end-to-end dentro de las restricciones de tiempo y equipo de un proyecto académico de 17 semanas, sin introducir vulnerabilidades de inyección SQL?

RQ3. ¿En qué medida el proceso de ingeniería de requisitos aplicado (elicitación, negociación, documentación y validación según el marco de Pohl [3], con las características de calidad INCOSE v4 [12]) produce un corpus de requisitos verificable y trazable de extremo a extremo hacia el código, las pruebas automatizadas y la evidencia empírica?

RQ4. ¿Qué amenazas a la validez de constructo, interna, externa y de conclusión afectan la generalización de los resultados empíricos reportados, y cómo se mitigaron dentro del alcance del proyecto?

1.3. Objetivo general y objetivos específicos

Objetivo general. Diseñar, implementar y evaluar empíricamente Artisync, una plataforma web de intermediación entre creadores de contenido digital y clientes, con una estrategia híbrida de acceso a datos y verificación de identidad asistida por inteligencia artificial, documentando el proceso completo con el rigor exigido por la ingeniería de software empírica.

Objetivos específicos:

1. Especificar los requisitos funcionales y no funcionales del sistema conforme a ISO/IEC/IEEE 29148:2018 [2] y validar su calidad contra las características C1–C15 de INCOSE v4 [12]. (*traza a RQ3*)
2. Implementar la estrategia híbrida de acceso a datos, conectando end-to-end al código en ejecución al menos seis procedimientos almacenados o funciones SQL categorizados por tipo de operación (consultas multi-tabla, cálculos agregados, reportes, actualizaciones masivas, validaciones cruzadas, generación de códigos secuenciales). (*traza a RQ2*)
3. Evaluar empíricamente el rendimiento bajo carga (k6), la cobertura de pruebas (JaCoCo) [13, 14], la calidad web (Lighthouse) y la seguridad (controles OWASP + análisis estático y dinámico automatizado) [4, 15–17] del sistema entregado, con datos crudos reproducibles. (*traza a RQ1*)

4. Evaluar la usabilidad percibida del sistema mediante el instrumento System Usability Scale [5,6] con una muestra de participantes externos al equipo de desarrollo. (*traza a RQ1*)
5. Documentar de forma explícita las amenazas a la validez de los resultados reportados y las decisiones de diseño no resueltas al cierre de la Entrega Final. (*traza a RQ4*)

1.4. Contribuciones

1. **Un sistema software completo y desplegable** (backend Spring Boot 4.1.0 / Java 21 LTS, frontend Angular, PostgreSQL 16, Redis 7, orquestado con Docker Compose) que implementa el ciclo completo de intermediación descrito en la Sección 1 — ver `artisync/`.
2. **Una estrategia híbrida de acceso a datos verificada end-to-end**, con 13 procedimientos/funciones PL/pgSQL versionados y 7 de ellos conectados al código Java en ejecución mediante JPA 2.1 sin concatenación de SQL dinámico — ver `db/procs/`, `docs/basedatos/CATALOGO-SP.md`, `docs/adr/adr-006-estrategia-acceso-datos.md`.
3. **Un corpus de ingeniería de requisitos trazable**, con 37 requisitos (23 funcionales, 14 no funcionales) especificados conforme a ISO/IEC/IEEE 29148:2018, historias de usuario en formato Connextra con criterios INVEST, casos de uso en plantilla de Cockburn [18], y una matriz de trazabilidad requisito → historia → caso de uso → módulo → endpoint → prueba → evidencia — ver `docs/requisitos/`, `docs/trazabilidad/matriz.csv`.
4. **Un paquete de evidencia empírica reproducible y versionada**, cubriendo rendimiento, seguridad, usabilidad, cobertura de código y calidad web, con datos crudos, scripts de análisis y diccionario de variables — ver `docs/mediciones/`.
5. **Un conjunto de siete registros de decisiones de arquitectura (ADR)** siguiendo la plantilla de Nygard [19], documentando y justificando las decisiones estructurales del sistema — ver `docs/adr/`.

1.5. Organización del documento

El Capítulo 2 presenta el marco teórico estrictamente necesario para comprender las decisiones de diseño e ingeniería de requisitos del proyecto. El Capítulo 3 sitúa el trabajo frente a la literatura de plataformas de intermediación y economía creativa. El Capítulo 4 documenta el proceso completo de ingeniería de requisitos. El Capítulo 5 describe la

metodología de Design Science Research aplicada y los instrumentos de medición empírica. El Capítulo 6 presenta el diseño arquitectónico del sistema y las decisiones registradas en los ADR. El Capítulo 7 detalla decisiones críticas de implementación, con énfasis en la estrategia híbrida de acceso a datos. El Capítulo 8 reporta los resultados de la evaluación empírica por cada dimensión de calidad. El Capítulo 9 discute esos resultados respondiendo a cada pregunta de investigación. El Capítulo 10 analiza las amenazas a la validez. El Capítulo 11 describe el trabajo futuro y el Capítulo 12 sintetiza las conclusiones. El documento cierra con las declaraciones obligatorias, la bibliografía y los anexos.

2. Marco teórico

Este capítulo se limita a los fundamentos estrictamente necesarios para comprender las decisiones documentadas en los capítulos posteriores; no es un compendio general de teoría de aplicaciones web.

2.1. Ingeniería de requisitos: ISO/IEC/IEEE 29148:2018 y el marco de Pohl

La norma ISO/IEC/IEEE 29148:2018 [2] define los procesos de ciclo de vida para la ingeniería de requisitos de sistemas y software, incluyendo la estructura recomendada de un Software Requirements Specification (SRS) y el patrón sintáctico de enunciado de requisito *[condición] [sujeto] shall [acción] [objeto] [restricción]*. Pohl [3] organiza el proceso en cuatro actividades interdependientes —elicitación, negociación, documentación y validación— que este proyecto usa como estructura del Capítulo 4. Wiegers y Beatty [20] complementan el marco con las prácticas de desarrollo y gestión de requisitos (priorización, control de cambios) usadas en `docs/requisitos/CHANGELOG-REQ.md`.

2.2. Calidad de requisitos: INCOSE Guide to Writing Requirements v4

La guía INCOSE v4 [12] define nueve características de calidad para un requisito individual (C1–C9: Necessary, Appropriate, Unambiguous, Complete, Singular, Feasible, Verifiable, Correct, Conforming) y seis para un conjunto de requisitos (C10–C15: Complete, Consistent, Feasible, Comprehensible, Able to be validated, Correct). Este proyecto aplica ambos conjuntos de características en `docs/checklists/incose-checklist.md`, con evidencia citada requisito por requisito en vez de una declaración genérica de cumplimiento.

2.3. Historias de usuario y casos de uso como técnicas complementarias

Las historias de usuario en formato Connextra (*As a [rol], I want [funcionalidad], so that [beneficio]*) con criterios INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable) capturan la intención funcional desde la perspectiva del usuario, mientras que los casos

de uso en la plantilla de Cockburn [18] formalizan el flujo de interacción en niveles 1 a 4 (resumen, cometido de usuario, subfunción, implementación). El proyecto usa ambas técnicas de forma complementaria, no sustitutiva: las historias en `docs/requisitos/historias/` capturan el valor de negocio; los casos de uso en `docs/requisitos/casos-de-uso/` capturan el flujo detallado de interacción para los escenarios críticos.

2.4. Modelo arquitectónico C4

El modelo C4 de Brown [21] describe la arquitectura de un sistema en cuatro niveles progresivos de detalle —Contexto, Contenedores, Componentes y Código— cada uno dirigido a una audiencia distinta (desde stakeholders no técnicos hasta desarrolladores). Este proyecto documenta los niveles 1 a 3 en `docs/diagramas/` (`C4_Nivel1_Contexto.md`, `C4_Nivel2_Contenedores.md`, `C4_Nivel3_Componentes_Backend.md`), usados como base del Capítulo 6.

2.5. Calidad de software: ISO/IEC 25010

ISO/IEC 25010:2011 [7] define un modelo de características de calidad de producto software (adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad, portabilidad) usado en este proyecto para clasificar los requisitos no funcionales del SRS y para estructurar la tabla de atributos de calidad del Capítulo 6. Las dimensiones evaluadas empíricamente en el Capítulo 8 (rendimiento, seguridad, usabilidad) se mapean directamente a subcaracterísticas de este modelo.

2.6. Principios REST

La arquitectura REST (Representational State Transfer), formalizada por Fielding en su disertación doctoral [22], define un conjunto de restricciones arquitectónicas (interfaz uniforme, sin estado en el servidor, cacheable, sistema en capas) que la API de Artisync sigue para exponer sus recursos (`/api/v1/...`), documentada con OpenAPI (Springdoc).

2.7. Autenticación: JWT (RFC 7519)

JSON Web Token (RFC 7519) [23] especifica un formato compacto y auto-contenido para representar afirmaciones (*claims*) entre dos partes, firmado digitalmente para garantizar integridad. Artisync usa JWT firmado HS256 para el access token; la decisión de dónde alojarlo (cuerpo de respuesta vs. cookie `HttpOnly`) y sus alternativas descartadas se documentan en `docs/adr/adr-002-esquema-autenticacion.md`.

2.8. Controles OWASP Top 10

El OWASP Top 10:2021 [4] cataloga las diez categorías de riesgo de seguridad más críticas en aplicaciones web, resultado de un análisis de datos de la industria. Este proyecto evidencia seis de esas categorías (A01 Control de acceso roto, A02 Fallas criptográficas, A03 Inyección, A05 Configuración de seguridad incorrecta, A07 Fallas de identificación y autenticación, A09 Fallas de registro y monitoreo de seguridad) con evidencia reproducible en `docs/mediciones/sec/`, complementadas con un escaneo automatizado OWASP ZAP baseline y análisis estático SpotBugs + find-sec-bugs.

2.9. Patrones de acceso a datos y modelos de consistencia

La disyuntiva entre mapeo objeto-relacional (ORM) y acceso directo mediante SQL/procedimientos almacenados es un compromiso reconocido en la literatura de arquitectura de datos y de ingeniería de software empírica sobre seguridad: el ORM reduce el código repetitivo de las operaciones CRUD elementales a costa de perder control fino sobre consultas complejas, mientras que los procedimientos almacenados ofrecen ejecución más cercana a los datos y una superficie de ataque de inyección SQL distinta (mitigable con parametrización estricta). La literatura fundacional sobre detección y prevención de inyección SQL — tanto por chequeo estático de consultas generadas dinámicamente [17] como por monitoreo en tiempo de ejecución de un modelo de consultas legítimas [16] — coincide en que el riesgo real no es el mecanismo elegido (ORM vs. procedimiento almacenado) sino la concatenación de entrada de usuario en el texto del comando SQL, sin importar cuál de los dos mecanismos la introduzca. La guía OWASP de prevención de inyección SQL [15] reconoce ambos mecanismos —consultas parametrizadas vía ORM y procedimientos almacenados correctamente parametrizados— como defensas primarias equivalentes contra la inyección, siempre que ninguno de los dos concatene entrada de usuario en el texto del comando SQL. Esta equivalencia es la base técnica de la estrategia híbrida adoptada en `docs/adr/adr-006-estrategia-acceso-datos.md` y evaluada en el Capítulo 8.

2.10. Documentación de decisiones de arquitectura (ADR)

El formato de *Architecture Decision Record* propuesto por Nygard [19] — un documento corto por decisión, con contexto, decisión y consecuencias — se adoptó en este proyecto precisamente por la razón que motivó su creación: los documentos grandes de arquitectura

no se mantienen actualizados, mientras que los documentos pequeños y modulares sí tienen una oportunidad real de actualizarse. Los siete ADR de Artisync (`docs/adr/`) siguen esta plantilla; el Capítulo 6 resume su contenido.

3. Trabajos relacionados

3.1. Declaración honesta de alcance

Este capítulo **no presenta una revisión sistemática de literatura** siguiendo el procedimiento completo de Kitchenham y Charters [24] ni el diagrama de flujo PRISMA 2020 [25]. Esa brecha ya está declarada de forma explícita y justificada en `docs/checklists/prisma-2020-checklist.md`, que concluye correctamente “No Aplica” porque el proyecto no ejecutó una revisión sistemática de literatura como parte de su metodología (ver Capítulo 5, Design Science Research de Peffers [1], que no exige una revisión sistemática como fase obligatoria). Presentar aquí una tabla comparativa de ocho filas con una cadena de búsqueda y criterios de inclusión/exclusión que nunca se ejecutaron constituiría una afirmación de completitud falsa — el mismo error que `docs/observaciones/INFORME-BRECHAS-ENTREGA-FINAL.md` identificó y corrigió en los checklists de la Fase 1 de esta entrega.

Para esta versión del documento se amplió la búsqueda de referencias contextuales (ver `docs/informe-final/README.md`, sección “Estado de la bibliografía”, para el detalle de las búsquedas ejecutadas), lo que permitió verificar contra fuente primaria e incorporar referencias adicionales sobre plataformas multilaterales, pagos en garantía y arquitectura de microservicios. Esto **no sustituye** una revisión sistemática: son adiciones puntuales dirigidas por los temas del proyecto, no el resultado de una cadena de búsqueda reproducible sobre bases de datos bibliográficas con criterios de inclusión/exclusión predefinidos.

Recomendación para el equipo antes del cierre: si el tiempo lo permite, ejecutar una búsqueda reducida (2–3 bases de datos, ventana temporal 2020–2026, 10–15 términos clave sobre “plataformas de intermediación digital”, “economía creativa”, “patrón escrow”, “verificación de identidad asistida por IA”) y completar este capítulo con al menos las ocho referencias que exige el Bloque B.6 de la guía, documentando el proceso real de búsqueda en vez de uno reconstruido a posteriori. Este borrador deja la estructura lista para ese trabajo.

3.2. Contexto mínimo defendible sin revisión sistemática

Sin pretender sustituir la revisión sistemática pendiente, el proyecto sí se sitúa frente a líneas de trabajo previamente publicadas que informaron decisiones de diseño concretas,

referenciadas también en el Capítulo 1:

- **Economía de creadores y plataformas de trabajo digital.** La literatura reciente documenta el crecimiento y las tensiones estructurales de la economía de creadores como categoría de estudio propia [8], y los retos operativos específicos de los freelancers que dependen de plataformas digitales fragmentadas [9] — el problema que motiva a Artisync en el Capítulo 1.
- **Plataformas multilaterales (*multi-sided platforms*).** El modelo Creador/Cliente de Artisync es, en la taxonomía de Hagiu y Wright [11], una plataforma que orquesta transacciones entre dos grupos de usuarios que de otro modo dependerían de intermediarios genéricos; esta literatura económica informa la decisión de negocio (no la técnica) de operar como intermediario auditable en vez de como marketplace pasivo.
- **Desarrollo de plataformas de comercio electrónico B2C/C2C.** Trabajos previos sobre el proceso de desarrollo de plataformas de e-commerce —caso de estudio de una ferretería B2C [26], o propuestas de mejora del proceso de desarrollo de plataformas de comercio electrónico con estudio de caso exploratorio en Chile [27]— informaron decisiones de proceso de este proyecto (metodología DSR, documentación arquitectónica C4), aunque ninguno aborda el patrón *escrow* ni la verificación de identidad asistida por IA específicamente.
- **Rol dual del vendedor/comprador en mercados digitales.** El trabajo de Wulfert y Schütte sobre el doble rol del minorista en mercados digitales [28] es relevante al modelo de negocio de doble lado (Creador/Cliente) de Artisync, documentado en docs/requisitos/SRS.md §2.3.
- **Pagos en garantía verificables.** La formalización de protocolos de depósito dual como mecanismo de *escrow* sin mediador confiable [10] ilustra, desde la literatura de sistemas distribuidos, la misma garantía transaccional que Artisync persigue por otra vía (PayPal Orders v2 con retención de fondos gestionada en el propio backend, no un contrato inteligente); se cita como contraste de diseño, no como arquitectura adoptada.
- **Arquitectura de microservicios.** El estudio exploratorio de Wang, Kadiyala y Rubin sobre promesas y desafíos reales de microservicios en la industria [29] contextualiza, por contraste, la decisión documentada en ADR-001 de *no* adoptar microservicios para Artisync dentro del plazo de 17 semanas del proyecto (Capítulo 6).

3.3. Brecha identificada (*research gap*)

Ninguna de las referencias anteriores combina, en un mismo sistema evaluado empíricamente: (i) el patrón de pago en garantía (*escrow*) integrado con un flujo de trabajo de pedido por etapas configurables, (ii) verificación de identidad y de certificados profesionales asistida por inteligencia artificial, y (iii) una estrategia híbrida de acceso a datos (ORM + procedimientos almacenados) evaluada con evidencia empírica reproducible de rendimiento, seguridad y cobertura de código. Ese es el aporte que este trabajo declara en el Capítulo 1 §1.4, sujeto a la verificación bibliográfica completa que el equipo debe realizar antes del cierre — ver nota de honestidad en la Sección 3.1.

4. Ingeniería de requisitos

Este capítulo es propio y no se fusiona con el Capítulo 6 (Diseño). Documenta el proceso completo de ingeniería de requisitos que llevó a la especificación vigente en v1.0.0, siguiendo la disciplina de ISO/IEC/IEEE 29148:2018 [2] y el marco de Pohl [3] (§2.1).

4.1. Contexto y stakeholders

Siguiendo la técnica de *stakeholder onion* de Alexander (capas concéntricas de cercanía al sistema):

Cuadro 4.1: Interesados por capa de cercanía al sistema

Capa	Interesados	Interés principal
Núcleo (equipo operador)	Los cuatro integrantes del equipo de desarrollo	Entregar un sistema funcional, evaluable y mantenible dentro del plazo académico
Usuarios directos	Creador de contenido, Cliente registrado, Administrador (roles definidos en docs/requisitos/SRS.md §2.3)	Contratar/ofrecer servicios con garantías de pago, comunicación y verificación de identidad
Usuarios indirectos	Visitante anónimo (explora el catálogo sin registrarse)	Evaluar la plataforma antes de registrarse
Patrocinador / regulador académico	Docente-director del PFC	Verificar que el sistema cumple los criterios de la rúbrica de cada entrega
Proveedores externos	PayPal (pagos), servicio de IA de verificación, proveedor de almacenamiento de objetos (Azure Blob / local)	Disponibilidad y estabilidad de la integración
Competencia / referencia de mercado	Plataformas genéricas de freelance y redes sociales usadas hoy de forma fragmentada (Capítulo 1)	No son stakeholders directos, pero delimitan el estándar de usabilidad y seguridad esperado

4.2. Técnicas de elicitación aplicadas

Declaración honesta de brecha: la guía exige declarar explícitamente las técnicas de elicitación usadas con evidencia archivada bajo `docs/requisitos/elicitacion/` (notas, guiones de entrevista, prototipos de baja fidelidad). Esa carpeta no existe en el repositorio a la fecha de este documento — ver `docs/observaciones/INFORME-BRECHAS-ENTREGA-FINAL.md`, Bloque A.3. El corpus original de requisitos (`Entrega 1A.docx`, semana del 4 de junio de 2026, referenciado en `docs/requisitos/SRS.md` §1.4) se produjo mediante análisis de documentos y discusión interna del equipo sobre el dominio de la economía creativa, pero no se conservó evidencia granular (notas de sesión, guiones) organizada como técnica de elicitación formal. Esta es una acción pendiente declarada, no una omisión silenciosa: el equipo debe decidir si reconstruye la evidencia disponible (actas de reunión, capturas de discusión) antes del cierre, o si documenta explícitamente la ausencia como limitación metodológica en el Capítulo 10 (Amenazas a la validez).

4.3. Requisitos funcionales y no funcionales: categorización y priorización

El corpus completo está en `docs/requisitos/SRS.md` (37 requisitos: 23 funcionales REQ-F-001–REQ-F-023, 14 no funcionales REQ-NF-001–REQ-NF-014), priorizado con Must, Should, Could, Won't (MoSCoW):

Cuadro 4.2: Distribución de requisitos por prioridad MoSCoW

Prioridad	Cantidad	%
Must	29	78.4 %
Should	7	18.9 %
Could	1	2.7 %
Won't	0	0 %

Nota de conformidad C9 (INCOSE v4), declarada sin maquillaje: los 37 requisitos están redactados en prosa descriptiva estructurada (*rationale*, prioridad, criterio de aceptación, verificación, estado), **no** en el patrón sintáctico formal [*condición*] [*sujeto*] *shall* [*acción*] [*objeto*] [*restricción*] que exige explícitamente ISO/IEC/IEEE 29148:2018 y las 42 reglas de INCOSE v4 [2, 12]. `docs/checklists/incose-checklist.md` documenta este hallazgo (C9, no cumplida) como el de mayor esfuerzo pendiente entre los 15 criterios evaluados — reescribir el corpus completo al patrón formal, no un ajuste puntual.

4.4. Modelado de casos de uso y de historias de usuario

- **Historias de usuario** (formato Connextra, criterios INVEST, *acceptance criteria* en Gherkin): docs/requisitos/historias/ (HU-01 a HU-23), cada una trazada a al menos un REQ-F-NNN y a una prueba automatizada de aceptación.
- **Casos de uso** (plantilla de Cockburn [18], niveles 1–4): docs/requisitos/casos-de-uso/ (CU-01 a CU-23), cada uno trazado a un flujo y a al menos una prueba de integración.
- **Diagrama de casos de uso UML de alto nivel:** Se realizó un análisis estructurado de los casos de uso para representar el comportamiento funcional del sistema frente a sus distintos perfiles. A nivel general, se definió un modelo que ilustra la interacción de los principales actores (Administrador, Creador, Cliente, Moderador, Soporte y Auditor) con los siete grandes subsistemas o macro-procesos de Artisync. Adicionalmente, el análisis se desglosó en catorce diagramas de casos de uso de alto nivel (dos por cada módulo), los cuales detallan operaciones críticas como la autenticación 2FA, la validación de portafolios mediante Inteligencia Artificial, la retención de fondos en el sistema Escrow y la moderación comunitaria. Todos los modelos, contruidos bajo sintaxis UML/Mermaid, se encuentran anexados y documentados en el archivo docs/informe-final/diagramas_casos_de_uso.md.

4.5. Validación de requisitos

El procedimiento de validación aplicado combina dos mecanismos verificables en el repositorio:

1. **Prueba automatizada de aceptación:** cada requisito *Must/Should* tiene un campo *Verificación* explícito en el SRS (Test/análisis/demostración) y una prueba automatizada referenciada en docs/trazabilidad/matriz.csv (columna prueba_automatizada).
2. **Estado verificado en la matriz de trazabilidad:** un requisito se marca verificado solo cuando su prueba automatizada está en verde contra el código en ejecución, no por declaración del equipo — la re-auditoría de docs/observaciones/INFORME-BRECHAS-ENTREGA-FINAL.md confirmó esta disciplina detectando y corrigiendo el único caso donde el estado no coincidía con la evidencia real (REQ-F-006/007, ver docs/checklists/incose-checklist.md, hallazgo C8).

Resultado de la validación final (2026-08-17, contra `matriz.csv`):

Cuadro 4.3: Estado de validación de los 37 requisitos

Estado	Requisitos	%
Verificado	25	67.6 %
Implementado (no verificado aún)	8	21.6 %
Pendiente	4	10.8 %

Los 4 requisitos *Pendiente*: REQ-F-009, REQ-F-010, REQ-NF-005, REQ-NF-006.

Los 29 requisitos *Must* están en estado implementado o verificado (ninguno pendiente); los 4 *Pendiente* corresponden a requisitos *Should/Could* — funciones sociales (seguidores, comentarios) y una medición de latencia WebSocket no ejecutada formalmente, ambos justificables como trabajo futuro (Capítulo 11) sin bloquear el cierre de los requisitos obligatorios.

4.6. Gestión del cambio de requisitos

`docs/requisitos/CHANGELOG-REQ.md` registra 2 versiones (`v0.3.0` → `v0.9.0-rc`: renombrado de identificadores `RF-NN` → `REQ-F-ONN`, sin cambio semántico). **Tasa de estabilidad declarada con salvedad** (ver `docs/checklists/incose-checklist.md`, métricas de calidad): nominalmente 100 % (0 modificaciones semánticas registradas), pero `matriz.csv` refleja cambios de estado reales (ej. REQ-F-023 pasó de *pendiente* a *verificado*) sin la entrada correspondiente en el changelog — la tasa de 100 % probablemente sobreestima la estabilidad real por subregistro, no por ausencia genuina de cambios. Corregir este subregistro es una acción de bajo esfuerzo pendiente para el cierre.

4.7. Métricas de calidad de requisitos

Ver el detalle completo, con evidencia citada característica por característica, en `docs/checklists/incose-checklist.md`. Resumen:

- **Número total de requisitos:** 37 (23 funcionales, 14 no funcionales).
- **Distribución por tipo:** funcionales 62.2 % (23/37), no funcionales 37.8 % (14/37).
- **Distribución por prioridad MoSCoW:** ver Tabla 4.2.
- **Porcentaje verificado:** 67.6 % (25/37); implementado o verificado (no pendiente): 89.2 % (33/37).

- **Cobertura de trazabilidad hacia código y pruebas:** 100 % de los 37 requisitos tiene fila en `matriz.csv` con módulo de código y prueba automatizada referenciados (columnas `modulo_codigo`, `prueba_automatizada`), confirmado por `scripts/validate-traceability.sh` en CI.
- **Características INCOSE v4 cumplidas:** 9 de 15 (C1, C2, C4, C6, C7, C11, C12, C13, C14); 5 parciales con evidencia concreta (C3, C8, C10, C15, C2 parcial); 2 no cumplidas (C5, C9) — ver detalle y plan de acción en el checklist referenciado.

5. Materiales y métodos

5.1. Enfoque metodológico global

El proyecto sigue la metodología Design Science Research (DSR) de Peffers et al. [1] en sus seis actividades canónicas, instanciadas así:

Cuadro 5.1: Instanciación de las seis actividades DSR de Peffers et al.

Actividad DSR	Instanciación en Artisync
1. Identificación del problema y motivación	Capítulo 1 — fragmentación de la economía creativa digital, ausencia de garantías de pago/identidad
2. Definición de objetivos de una solución	Objetivos específicos del Capítulo 1 §1.3, derivados de las RQ
3. Diseño y desarrollo	Capítulos 6 (arquitectura) y 7 (implementación); cuatro entregas incrementales (1A→1B→3→Final)
4. Demostración	Sistema desplegado y operativo, con usuario demo documentado (<code>README.md</code>)
5. Evaluación	Capítulo 8 — evaluación empírica multidimensional (rendimiento, seguridad, usabilidad, cobertura, calidad web)
6. Comunicación	Este documento, el repositorio público con DOI Zenodo, y la defensa oral del PFC

El propio marco DSR de Peffers et al. se apoya en los siete lineamientos de rigor de Hevner, March, Park y Ram [30] para investigación en sistemas de información (relevancia del problema, rigor de diseño, evaluación del artefacto, contribución a la investigación, rigor de investigación, diseño como proceso de búsqueda, comunicación de resultados), que este proyecto usa como criterio implícito de calidad al decidir qué evidencia archivar en `docs/mediciones/`.

Dado que el trabajo aporta evaluación empírica de rendimiento y de usabilidad de un artefacto único (no un experimento controlado con grupos de comparación aleatorizados ni un estudio de caso sobre una organización externa), se acoge además el estándar empírico *Engineering Research* de Ralph et al. [31] para el reporte metodológico — ver checklist completo en `docs/checklists/ralph-2021-checklist.md`. Se consultaron adicionalmente las guías de reporte de estudios de caso de Runeson y Höst [32] como referencia de rigor

metodológico para documentar el protocolo experimental (§5.5), aunque Artisync no se reporta formalmente como un estudio de caso en el sentido estricto de esa guía (no hay una organización externa como unidad de análisis).

5.2. Instrumentos de medición

Cuadro 5.2: Instrumentos de medición empírica utilizados

Instrumento	Versión exacta	Qué mide	Configuración
SUS de Brooke [5, 6]	10 ítems originales, sin modificación	Usabilidad percibida	Formulario Google Forms, escala 1–5, ver docs/mediciones/sus/instrucciones-formulario.md
k6	v2.1.0 (go1.26.4)	Rendimiento bajo carga	50 VUs, 30 s, k6/catalogo-load.js (ver §5.5)
Lighthouse / @lhci/cli	Lighthouse 12.6.1 / @lhci/cli 0.15.1	Calidad web (Performance, Accessibility, Best Practices, SEO)	lighthouseci.mobile.json / lighthouseci.desktop.json, 3 corridas por perfil
JaCoCo [13, 14]	0.8.13 (jacoco-maven-plugin)	Cobertura de código (líneas, ramas, complejidad)	pom.xml, umbral declarado $\geq 70\%$
OWASP ZAP [4]	ghcr.io/zaproxy/zaproxy:stable (zap-baseline.py)	Escaneo dinámico de seguridad	Modo baseline contra http://localhost:4200
SpotBugs + find-sec-bugs [16, 17]	4.8.6.6 + 1.13.0	Análisis estático de inyección SQL	Regla SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE y variantes

Versiones exactas de todo el entorno (Docker, JDK, Node, Angular CLI) en docs/entorno/versions.txt.

5.3. Enfoque de métricas: Goal-Question-Metric (GQM)

Ejemplo de GQM completo aplicado al bloque de rendimiento, siguiendo el enfoque original de Basili, Caldiera y Rombach [33]:

- **Meta (Goal):** Analizar el rendimiento del endpoint de catálogo con el propósito de caracterizarlo desde el punto de vista de la latencia percibida por el usuario, en el contexto del sistema Artisync bajo carga concurrente moderada.
- **Preguntas (Questions):**
 - Q1: ¿Cuál es la latencia p95 del endpoint bajo 50 usuarios virtuales concurrentes?
 - Q2: ¿Se mantiene esa latencia dentro del umbral declarado (200 ms con caché caliente, 500 ms con caché frío)?
 - Q3: ¿Existen errores de servidor (HTTP ≥ 500) bajo esa carga?
- **Métricas (Metrics):**
 - M1: percentil 95 de `http_req_duration` (ms), por escenario.
 - M2: tasa de `http_req_failed` con código ≥ 500 (%).
 - M3: throughput agregado (`http_reqs/s`).

El mismo patrón GQM (meta declarada en el `REPORTE-*.md` de cada bloque, preguntas implícitas en los umbrales del SRS, métricas reportadas con estadística descriptiva) se repite para seguridad, usabilidad, cobertura y calidad web — ver `docs/mediciones/<bloque>/REPORTE-*.md`.

5.4. Población, muestreo y participantes

Bloque de usabilidad (SUS): población objetivo — usuarios potenciales de una plataforma de intermediación de servicios digitales (perfil generalista, no exclusivamente técnico). Muestra: **N=16 participantes externos al equipo de desarrollo**, reclutados por conveniencia (red de contactos del equipo), rango de edad 21–62 años ($M=34.6$), 50 % femenino / 50 % masculino, experiencia web 50 % alta / 31.3 % media / 18.8 % baja, dispositivo 37.5 % móvil / 25 % laptop / 25 % desktop / 12.5 % tablet (`docs/mediciones/sus/perfil-participantes.csv`).

Limitación de muestreo declarada honestamente (siguiendo Baltes y Ralph [34] sobre reporte de muestreo en ingeniería de software empírica): el muestreo por conveniencia no garantiza representatividad de la población de usuarios reales de la plataforma

(creadores y clientes de economía creativa específicamente), y **no existe una justificación de tamaño de muestra basada en poder estadístico** — N=16 se decidió por disponibilidad de participantes dentro del plazo académico, no por un cálculo a priori. Esta limitación se retoma en el Capítulo 10 (Amenazas a la validez externa).

Bloques automatizados (k6, Lighthouse, JaCoCo, OWASP/ZAP): no aplican muestreo de sujetos humanos; el “tamaño de muestra” es el número de corridas independientes (3 para Lighthouse por perfil, 3+3 para k6 por escenario caliente/frío, 1 ejecución exhaustiva para JaCoCo sobre la suite de 522 pruebas).

5.5. Protocolo experimental replicable

Cada bloque de evaluación tiene un comando reproducible documentado en `Makefile` y su reporte:

- **Rendimiento:** `make bench` (requiere `make up` y `k6` instalado) — ejecuta `k6/catalogo-load.js` contra `GET /api/v1/catalogo?page=0&size=20,50 VUs/30s`. Ver `docs/mediciones/perf/REPORTE-PERF.md` para el protocolo detallado de los escenarios caliente y frío.
- **Seguridad:** `make audit` (análisis estático) + `make audit-zap` (escaneo dinámico) — ver `docs/mediciones/sec/REPORTE-SEC.md`.
- **Usabilidad:** protocolo de tarea guiada (registro/login → explorar catálogo → crear pedido → revisar seguimiento → cerrar sesión) descrito en `docs/mediciones/sus/instrucciones-formulario.md`, seguido del cuestionario SUS; análisis con `make sus`.
- **Cobertura:** `make test` (JUnit 5 + JaCoCo, reporte en `docs/mediciones/jacoco/`).
- **Calidad web:** `make lighthouse` — ver detalle de la configuración del contenedor efímero en el propio `Makefile` (razones técnicas del aislamiento de red documentadas ahí).

5.6. Análisis estadístico

Estadísticos calculados en todas las mediciones cuantitativas: **media, desviación típica e intervalo de confianza al 95 %**, preferidos sobre valores p aislados como forma primaria de reporte, siguiendo la recomendación de la comunidad de ingeniería de software empírica [35,36]. Ver valores concretos por bloque en el Capítulo 8.

Brecha declarada honestamente: la guía de la Entrega Final exige, cuando se comparan dos condiciones (ej. caché frío vs. caliente en el bloque de rendimiento), un test inferencial no paramétrico (Wilcoxon, Mann-Whitney) acompañado de un tamaño de efecto (*Cliff's delta*, r de rangos) — exactamente el procedimiento que Arcuri y Briand [36] sistematizan para algoritmos y comparaciones no deterministas en ingeniería de software. **Ese test no se ejecutó** — `docs/checklists/ralph-2021-checklist.md` y `docs/checklists/fair-checklist.md` documentan esta ausencia sin maquillarla. Además, la propia comparación caliente/frío tiene una limitación metodológica de diseño documentada en `docs/mediciones/perf/REPORTE-PERF.md`: el script de carga no aísla un *cache miss* real por iteración, por lo que un test inferencial sobre los datos actuales no respondería a la pregunta que pretende responder. Corregir el diseño del experimento (variar los parámetros de consulta para forzar *misses* reales) es prerequisite para que el test inferencial tenga sentido — declarado como trabajo futuro en el Capítulo 11.

No se realizaron comparaciones múltiples que requieran corrección (Holm, Benjamini-Hochberg) en esta entrega.

5.7. Determinismo y semillas

Ver el detalle completo en `docs/entorno/versions.txt`, sección “Semillas aleatorias”. Resumen: `fn_seleccionar_ganadores_sorteo` usa `random()` de PostgreSQL sin semilla fija (no determinista por diseño — es un sorteo real); los scripts de análisis SUS no usan muestreo aleatorio. No se identificaron otras dependencias de semillas aleatorias en los scripts de medición a la fecha de este documento.

6. Diseño del sistema y arquitectura

6.1. Modelo C4

6.1.1. Nivel 1 — Contexto

Artisync se sitúa como sistema central que conecta tres actores (Cliente, Creador/Artista, Administrador) con tres sistemas externos: PayPal (pagos, patrón *escrow*), un servicio de IA de verificación de identidad/certificados, y un proveedor de almacenamiento de objetos (Azure Blob Storage o local, según ADR-007). El modelo C4 usado como base sigue a Brown [21]. Diagrama completo: `docs/diagramas/C4_Nivel1_Contexto.md` y `C4-nivel1-context_diagram.svg`.

6.1.2. Nivel 2 — Contenedores

Cuatro contenedores desplegados vía Docker Compose: **frontend** (Angular, SPA servida por nginx), **backend** (Spring Boot, API REST documentada con OpenAPI/Springdoc), **postgres** (PostgreSQL 16, esquema versionado con Flyway) y **redis** (caché de catálogo, blacklist de JWT revocados, rate limiting, tickets 2FA). Diagrama completo: `docs/diagramas/C4_Nivel2_Contenedores.md`.

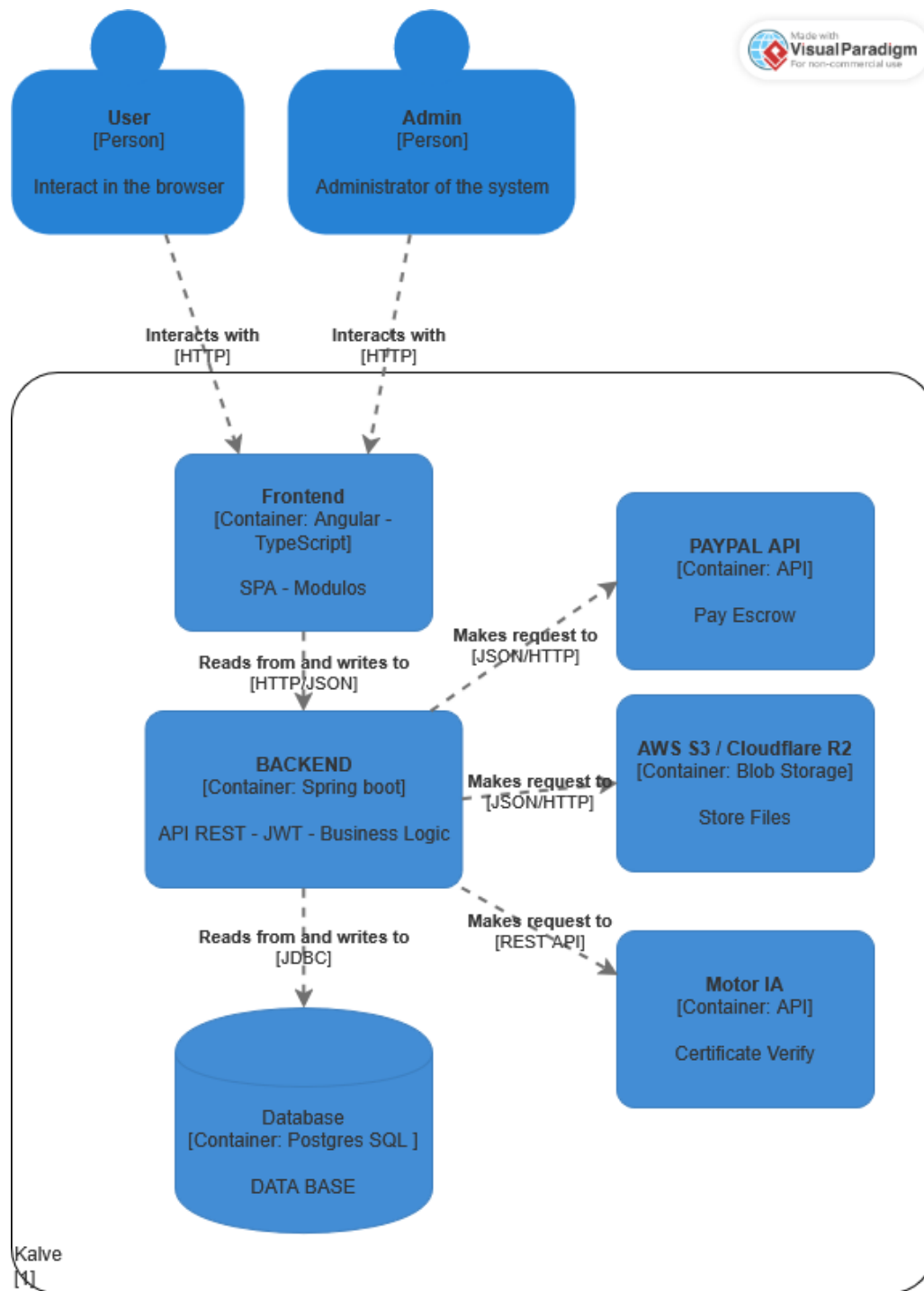


Figura 6.1: Diagrama C4 Nivel 2 — Contenedores de Artisync. Fuente: docs/diagramas/c4-nivel2-contenedores.png.

Nota de honestidad sobre versiones documentadas vs. reales: los diagramas C4 (y ADR-001) documentan la pila como “Java 25 + Spring Boot 4.0.1” y “PostgreSQL 18/Angular 22” en su versión original. La pila **realmente construida y desplegada**, verificada contra pom.xml, los Dockerfile y docker-compose.yml a la fecha de este documento, es: **Java 21 LTS** (target de bytecode) + **Spring Boot 4.1.0**, compilado en contenedor maven:3.9-eclipse-temurin-21 y ejecutado en eclipse-temurin:21-jre-alpine;

Angular ^22.0.0 sobre **node:24-alpine**; **PostgreSQL 16** (**postgres:16**, anclado por digest sha256) y **Redis 7** (**redis:7-alpine**, igualmente anclado). La documentación arquitectónica no se actualizó en cascada cuando el equipo fijó las versiones finales — ver `docs/entorno/versions.txt` para la versión verificada y autoritativa, que este capítulo usa como fuente en vez de repetir la cifra desactualizada de los diagramas.

6.1.3. Nivel 3 — Componentes (backend)

El backend se organiza en siete módulos funcionales por dominio (**seguridad**, **perfil**, **catalogo**, **pedido/flujo de trabajo**, **legal**, **comunicacion**, **social**), cada uno con sus capas `entity/dto/repository/service/controller`. Detalle completo: `docs/diagramas/C4_Nivel3_Componentes_Backend.md`.

6.2. Resumen de los siete registros de decisión de arquitectura (ADR)

Los siete ADR siguen la plantilla de Nygard [19]:

Cuadro 6.1: Resumen de los siete ADR de Artisync

ADR	Título	Decisión resumida
ADR-001	Selección de la tecnología principal	Java/Spring Boot en el backend (mayor throughput, seguridad/persistencia integradas), Angular/TypeScript en el frontend, PostgreSQL como motor relacional, Redis como caché, Docker Compose como orquestador — ver nota de versiones §6.1.
ADR-002	Esquema de autenticación y control de acceso	JWT (HS256) devuelto en el cuerpo de la respuesta para el access token (no en cookie); refresh token y ticket pre-auth de 2FA en cookies <code>HttpOnly + SameSite=Strict</code> ; blacklist de JTI en Redis; bcrypt factor 12; RBAC vía Spring Security. Revisado en agosto 2026 tras corregir una afirmación incorrecta de una versión anterior del ADR (ver texto completo).
ADR-003	Selección del gestor de base de datos	PostgreSQL 16 sobre MySQL/MariaDB y MongoDB, por ACID completo, tipos DECIMAL precisos para montos financieros, y soporte maduro de PL/pgSQL para la estrategia híbrida de acceso a datos (ADR-006).
ADR-004	Estrategia de caché	Redis 7 para caché de catálogo (TTL configurable), blacklist de JTI, cuotas de rate limiting y tickets pre-auth 2FA, sobre caché en memoria de la JVM, por ser compartido entre réplicas si el backend escala horizontalmente.
ADR-005	Estrategia de despliegue y reproducibilidad	Docker Compose con imágenes ancladas por digest <code>sha256 + Makefile</code> de un solo comando, sobre despliegue manual o imágenes por tag variable, para máxima reproducibilidad (badge ACM <i>Artifacts Evaluated — Reusable</i>). Las tres acciones pendientes de la Tercera Entrega (Frontend/ ausente, digests por tag, Makefile inexistente) están resueltas y verificadas a la fecha de la Entrega Final.
ADR-006	Estrategia híbrida de acceso a datos	CRUD elementales vía JPA/Spring Data; operaciones con joins, agregaciones, reportes, actualizaciones masivas o validaciones cruzadas vía procedimientos/funciones PL/pgSQL versionados en <code>db/procs/</code> , invocados exclusivamente mediante JPA 2.1. Ampliado el 16 de agosto de 2026 con 7 rutinas nuevas conectadas end-to-end (ver Capítulo 7).
ADR-007	Almacenamiento de archivos en la nube	Azure Blob Storage como único destino en la nube (sobre un esquema híbrido Cloudinary+Azure Blob o mantener el volumen local), detrás de la interfaz <code>AlmacenamientoDocumentos</code> , seleccionable por con-

Texto completo de cada ADR, con contexto, opciones descartadas y consecuencias, en `docs/adr/`.

6.3. Atributos de calidad (ISO/IEC 25010) — prioridad, escenario y estrategia

Cuadro 6.2: Atributos de calidad ISO/IEC 25010 [7] priorizados

Característica ISO 25010	Prioridad	Escenario	Estrategia aplicada
Eficiencia de desempeño (rendimiento)	Alta	Listado de catálogo bajo 50 usuarios concurrentes	Caché Redis con TTL (ADR-004); índices en columnas de filtro; medido con k6 (Capítulo 8)
Seguridad	Alta	Acceso no autorizado a rutas protegidas, inyección SQL, credenciales en tránsito	RBAC + JWT (ADR-002); procedimientos/consultas parametrizadas (ADR-006); rate limiting; auditoría estática y dinámica automatizada (Capítulo 8)
Fiabilidad	Alta	Caída de un contenedor dependiente (Redis, Postgres)	<code>healthcheck</code> de Docker Compose con <code>depends_on: condition: service_healthy</code> ; <code>/actuator/health</code> expone estado por componente
Usabilidad	Media	Usuario nuevo completa el flujo de contratación sin asistencia	Formularios dinámicos, flujo de contratación acotado (SRS REQ-NF-008); medido con SUS (Capítulo 8)
Mantenibilidad	Media	Incorporar un nuevo procedimiento almacenado sin tocar el código Java existente	Módulos desacoplados por dominio (Nivel 3 C4); <code>scripts/sync-procs.sh</code> mantiene sincronizada la migración repetible
Portabilidad	Media	Desplegar en un proveedor cloud distinto al usado en desarrollo	Configuración externalizada (<code>.env</code>), <code>documentos.proveedor</code> seleccionable (ADR-007), imágenes ancladas por digest (ADR-005)
Compatibilidad	Baja	Integración con PayPal Orders v2 y servicio externo de IA	Contratos de API versionados, adaptadores dedicados por integración

ver Capítulo 7), el resto **CRUD-ORM**.

7. Implementación

No se transcriben archivos completos; se remite al repositorio para el detalle. Este capítulo ilustra con listados pequeños y significativos las decisiones críticas de código.

7.1. Manejo de errores globales (RFC 7807 Problem Details)

Todos los errores de la API se serializan como `ProblemDetail` (RFC 7807), en vez de estructuras JSON ad-hoc distintas por controlador.

Listing 7.1: Manejador global de excepciones (RFC 7807 Problem-Detail). Fuente: `artisync/Backend/src/main/java/.../exception/ManejadorGlobalExcepciones.java`

```
1  @Slf4j
2  @RestControllerAdvice
3  public class ManejadorGlobalExcepciones {
4
5      private static final String BASE_TIPO = "https://artisync.dev/errors/";
6
7      @ExceptionHandler(AccessDeniedException.class)
8      public ResponseEntity<ProblemDetail> manejarAccesoDenegado(AccessDeniedException ex
9          ) {
10         ProblemDetail problema = ProblemDetail.forStatus(HttpStatus.FORBIDDEN);
11         problema.setType(URI.create(BASE_TIPO + "acceso-denegado"));
12         problema.setTitle("Acceso denegado");
13         // ... (continua con los demas @ExceptionHandler: validacion, JWT, 404, etc.)
14     }
```

Esta decisión centraliza el formato de error para las 522 pruebas del backend y para el interceptor HTTP del frontend Angular, evitando el manejo de errores ad-hoc por controlador.

7.2. Esquema de caching

El listado de catálogo (endpoint de mayor frecuencia de lectura, REQ-NF-004) usa Spring Cache sobre Redis (ADR-004), con invalidación automática (`@CacheEvict`) al crear/editar/eliminar un servicio:

Listing 7.2: Caché del listado de catálogo (Spring Cache + Redis).

Fuente: `artisync/Backend/src/main/java/.../service/catalogo/impl/ServicioCatalogoServicioImpl.java`

```
1 @Override
2 @Transactional(readOnly = true)
3 @Cacheable(cacheNames = "catalogo")
4 public Page<RespuestaServicioResumido> buscarCatalogoServicios(
5     Long categoriaId, Long subcategoriaId, BigDecimal precioMin, BigDecimal
6     precioMax,
7     List<Long> etiquetaIds, String textoBusqueda, String sortParam, int page, int
8     size) {
9     // ... construye la Specification dinamica y ejecuta la consulta paginada
10 }
```

La clave de caché de Spring se compone de todos los parámetros del método — una limitación de diseño relevante para la interpretación de los resultados de rendimiento del Capítulo 8 (ver `docs/mediciones/perf/REPORTE-PERF.md`, advertencia metodológica sobre el escenario “frío”).

7.3. Mecanismo de seguridad: JWT + rate limiting

La autenticación es *stateless* (JWT [23], ADR-002); las rutas de autenticación aplican rate limiting por IP real (resuelta vía `server.forward-headers-strategy=native`, no `getRemoteAddr()` directo) y por cuenta, con respuesta 429 en formato `ProblemDetail` — ver `AuthRateLimitFilter` e `IntentosAutenticacionService`, corrección de la observación OBS-AUTO-05 documentada en `docs/observaciones/OBSERVACIONES.md`.

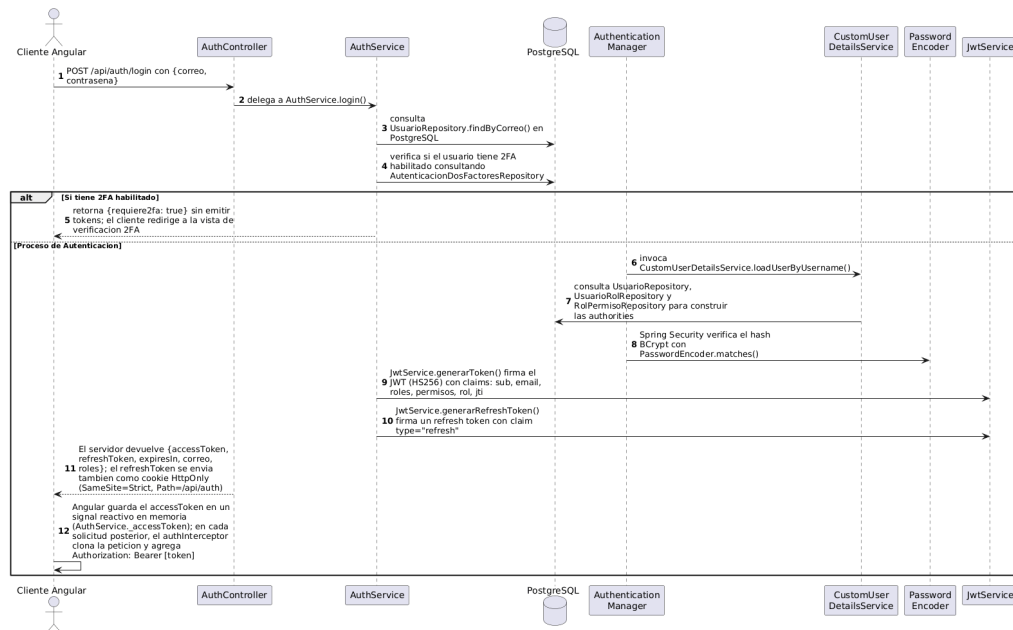


Figura 7.1: Diagrama de secuencia del flujo de autenticación JWT. Fuente: docs/diagramas/secuencia_login_jwt.png.

7.4. Estrategia híbrida de acceso a datos: procedimiento almacenado representativo

Esta subsección ilustra el contrato completo entre una función PL/pgSQL versionada en db/procs/ y el método de repositorio Spring Data que la invoca, siguiendo la especificación JPA 2.1 (ADR-006, Capítulo 6 §6.2).

Listing 7.3: fn_registrar_usuario (categoría: validaciones cruzadas + inserción multi-tabla). Fuente: db/procs/fn_registrar_usuario.sql — soporta REQ-F-001

```

1  -- Registra un usuario en una unica transaccion atomica: valida correo unico,
2  -- mayoria de edad (RNF-12, >=18 años) y rol permitido en auto-registro;
3  -- inserta en usuarios + usuario_rol + perfil de creador (si aplica).
4  -- El hash de contraseña llega ya calculado (BCrypt vive en Java, no en el motor).
5  -- Parametros formales tipados; sin concatenacion ni EXECUTE.
6  CREATE OR REPLACE FUNCTION fn_registrar_usuario(
7      p_nombres VARCHAR(100),
8      p_apellidos VARCHAR(100),
9      p_correo VARCHAR(150),
10     p_contraseña_hash VARCHAR(255),
11     p_fecha_nacimiento DATE,
12     p_nombre_rol VARCHAR(50)
13 ) RETURNS BIGINT LANGUAGE plpgsql AS $$
14 -- ... valida y ejecuta los INSERT atomicos (ver archivo completo en db/procs/)
15 $$;
```

Listing 7.4: Método de repositorio que invoca `fn_registrar_usuario`. Fuente: `artisync/Backend/src/main/java/.../repository/seguridad/UsuarioRepository.java`

```

1  /** REQ-F-001 - fn_registrar_usuario: inserta usuario + usuario_rol + perfil de
    creador
2      opcional. Devuelve el id_usuario generado. */
3  @Query(value = "SELECT fn_registrar_usuario(:p_nombres, :p_apellidos, :p_correo, "
4      + ":p_contrasena_hash, :p_fecha_nacimiento, :p_nombre_rol)", nativeQuery = true
5      )
6  Long registrarUsuario(
7      @Param("p_nombres") String nombres,
8      @Param("p_apellidos") String apellidos,
9      @Param("p_correo") String correo,
10     @Param("p_contrasena_hash") String contrasenaHash,
11     @Param("p_fecha_nacimiento") LocalDate fechaNacimiento,
12     @Param("p_nombre_rol") String nombreRol);

```

7.4.1. Nuance de conformidad, declarada sin maquillaje

La guía exige que la invocación use “los mecanismos formales de la especificación JPA 2.1 (@Procedure sobre método de repositorio Spring Data o @NamedStoredProcedureQuery sobre entidad)”. Las 7 rutinas de la ampliación de la Entrega Final (incluida `fn_registrar_usuario` arriba) están declaradas como **FUNCTION** de PostgreSQL (no **PROCEDURE**) e invocadas con `@Query(value = "SELECT fn_x(...)", nativeQuery = true)` — parametrizado con `@Param` nombrados, **sin concatenación de cadenas** (por lo que no se dispara la regla transversal 7 de la guía, que prohíbe específicamente la concatenación — ver el análisis fundacional de Gould, Su y Devanbu [17] y de Halfond y Orso [16] sobre por qué la concatenación, y no el mecanismo de invocación en sí, es la causa raíz de la inyección SQL), pero tampoco es literalmente `@Procedure` ni `@NamedStoredProcedureQuery`. El propio código lo documenta explícitamente en un comentario de `UsuarioRepository.java`: `@Procedure` en este repositorio solo se verificó contra un **CREATE PROCEDURE** real (`sp_registrar_decision_verificacion`, del módulo de verificación por IA, que sí es invocado con `@Procedure` estricto). El equipo debe decidir, antes del cierre, si (a) reescribe las 7 funciones como **PROCEDURE** para poder usar `@Procedure` literal, (b) usa `@NamedStoredProcedureQuery` sobre `SELECT fn_x(...)` (compatible con **FUNCTION**), o (c) declara explícitamente en el capítulo de Diseño por qué `@Query(nativeQuery=true)` parametrizado satisface el espíritu de la regla (invocación formal, sin concatenación) aunque no su letra exacta. Este hallazgo no estaba señalado en `docs/observaciones/OBSERVACIONES.md` antes de este documento — se recomienda añadirlo como observación nueva.

8. Evaluación empírica y resultados

Cada bloque se reporta con: (i) pregunta de investigación asociada, (ii) métrica, (iii) datos crudos y su ubicación, (iv) estadística descriptiva, y (v) interpretación textual.

8.1. Rendimiento (k6) — RQ1

Métrica: percentiles de latencia (`http_req_duration`) y tasa de error HTTP ≥ 500 sobre `GET /api/v1/catalogo?page=0&size=20`, 50 VUs, 30 s, 3 corridas por escenario. **Datos crudos:** `docs/mediciones/perf/k6-run{1,2,3}.json` (caliente), `k6-cold-run{1,2,3}.json` (frío); consolas en `k6-console-run*.txt`.

Cuadro 8.1: Resultados de la prueba de carga k6 por escenario

Escenario	n	Media (ms)	Mediana (ms)	DT (ms)	IC 95 % (ms)	p50/p90/p95/p99 (ms)	Error ≥ 500
Caliente	4500	21.38	13.01	32.71	[20.42, 22.33]	13.01 / 31.75 / 50.17 / 205.60	0.00 %
Frío	4500	13.62	9.04	14.52	[13.20, 14.05]	9.04 / 22.39 / 39.14 / 89.98	0.00 %

Umbral: $p_{95} \leq 200$ ms caliente (**cumple**, 50.17 ms), $p_{95} \leq 500$ ms frío (**cumple**, 39.14 ms). Throughput ≈ 48.5 –49.0 req/s por corrida.

Interpretación: ambos escenarios cumplen holgadamente sus umbrales respectivos. El hallazgo más relevante no es el cumplimiento en sí, sino que el escenario “frío” resultó **más rápido** que el “caliente” — contraintuitivo, y documentado con honestidad metodológica en `docs/mediciones/perf/REPORTE-PERF.md`: el script de carga golpea siempre la misma combinación de parámetros de consulta, por lo que la clave de caché de Spring (`@Cacheable`, Capítulo 7 §7.2) se puebla con la primera petición en ambos escenarios — de las 1500 iteraciones por corrida, como mucho una es un *miss* real contra PostgreSQL. La diferencia de medias observada (21.38 ms vs. 13.62 ms) es ruido de ejecución (JIT/GC de la JVM, carga del contenedor), no un efecto real de caché frío vs. caliente. **No se ejecutó** el test inferencial (Wilcoxon) ni el tamaño de efecto que exige el Bloque C de la guía para esta comparación — protocolo recomendado por Arcuri y Briand [36], declarado como brecha en el Capítulo 5 §5.6 y retomado como amenaza a la validez en el Capítulo 10.

8.2. Seguridad (OWASP + ZAP + análisis estático) — RQ1, RQ2

Métrica: estado (aprobado/hallazgo) de 6 controles OWASP [4] evidenciados manualmente, más el resultado agregado del escaneo automatizado ZAP baseline y del análisis estático SpotBugs + find-sec-bugs [16,17]. **Datos crudos:** docs/mediciones/sec/owasp/*.txt, sec/zap/, sec/static-analysis/.

Cuadro 8.2: Controles OWASP evidenciados manualmente

Control OWASP	Estado	Evidencia
A01 — Control de acceso roto	Aprobado	sec/owasp/a01-control-acceso.txt
A02 — Fallas criptográficas (TLS)	Aprobado	sec/owasp/a02-tls.txt
A03 — Inyección	Aprobado	sec/owasp/a03-inyeccion.txt
A05 — Configuración de seguridad incorrecta	Aprobado	sec/owasp/a05-cabeceras.txt
A07 — Fallas de identificación y autenticación	Aprobado	sec/owasp/a07-rate-limit.txt
A09 — Fallas de registro y monitoreo	Aprobado	sec/owasp/a09-logging.txt

A04, A06, A08, A10 quedan fuera de alcance declarado (no evaluados) — ver docs/mediciones/sec/REPORTE-SEC.md.

ZAP baseline: FAIL-NEW: 0 · WARN-NEW: 8 (severidad media/baja, ninguno bloqueante) · PASS: 59. Cumple el umbral de “sin hallazgos altos”.

Análisis estático (SpotBugs + find-sec-bugs): 0 hallazgos de inyección SQL (SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE, SQL_INJECTION_JDBC y variantes) sobre el bytecode del backend completo — consistente con el modelo de detección estático propuesto originalmente por Gould, Su y Devanbu [17].

Interpretación: la combinación de evidencia manual reproducible (6 controles) y escaneo automatizado (ZAP + análisis estático) da una cobertura de seguridad consistente en ambas direcciones (top-down por control OWASP, bottom-up por herramienta automatizada), sin contradicciones entre ambas fuentes.

8.3. Usabilidad (SUS) — RQ1

Métrica: puntaje System Usability Scale (0–100) [5], N=16 (umbral mínimo de la guía: $N \geq 15$). **Datos crudos:** docs/mediciones/sus/sus-raw.csv, perfil de participantes en perfil-participantes.csv.

Cuadro 8.3: Resultados del cuestionario SUS

Métrica	Valor
n	16
Media	76.88
Mediana	77.50
DT	14.48
IC 95 %	[69.16, 84.59]
Interpretación (escala de Bangor [6])	B
Umbral del proyecto (>68)	Supera

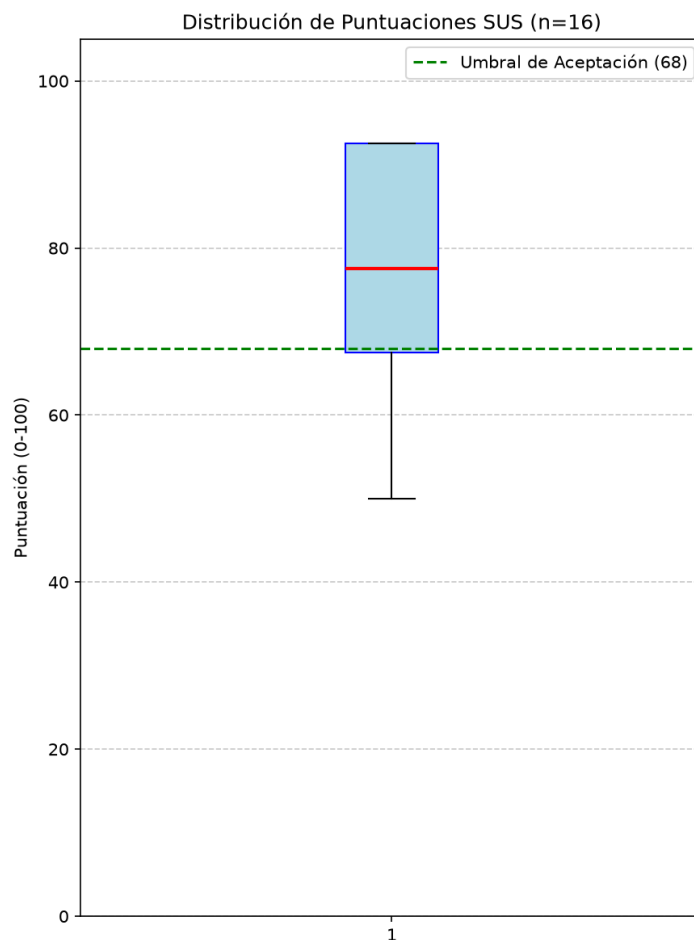


Figura 8.1: Diagrama de caja de los puntajes SUS ($N = 16$). Fuente: docs/mediciones/sus/boxplot-sus.png.

Nota de conformidad: el Bloque C de la guía exige gráficos vectoriales (SVG/PDF); la Figura 8.1 está en formato raster (PNG). Regenerarlo en SVG desde el mismo script (docs/mediciones/sus/graficar-sus.py) es una corrección de bajo esfuerzo pendiente antes del cierre.

Interpretación: el puntaje medio (76.88) está por encima del percentil 50 de la escala SUS normalizada y en la categoría B de Bangor (rango “excelente” bajo), superando el umbral mínimo del proyecto (>68) con margen. La limitación de muestreo por conveniencia (Capítulo 5 §5.4) restringe la generalización de este resultado a la población real de usuarios — retomado en el Capítulo 10.

8.4. Cobertura de código (JaCoCo) — RQ1

Métrica: cobertura de líneas, ramas y complejidad ciclomática sobre los módulos de dominio, servicios y controladores — las mismas tres dimensiones que Zhu, Hall y May [13] sistematizan como criterios de adecuación de pruebas. **Datos crudos:**

`docs/mediciones/jacoco/report.xml`, `html/jacoco.csv`.

Cuadro 8.4: Cobertura de código JaCoCo

Métrica	Cobertura	Umbral	Cumple
Lines	72.0 % (2867/3981)	≥ 70 %	✓
Branches	62.5 % (703/1125)	≥ 70 %	×
Instructions	69.7 % (12825/18404)	—	—
Complexity	56.5 % (775/1371)	—	—

Interpretación: la cobertura de líneas cumple el umbral de 70 %; **la cobertura de ramas no lo alcanza** (62.5 % vs. 70 %) — declarado sin maquillaje, es una brecha real, no un redondeo favorable. Cabe además la advertencia empírica de Inozemtseva y Holmes [14]: un porcentaje de cobertura alto no implica por sí solo una suite de pruebas efectiva para detectar defectos, por lo que estas cifras deben leerse como una medida de exhaustividad estructural, no como un sustituto de la efectividad real de las 522 pruebas. La medición es un agregado global del backend; la guía exige desglose por módulo (dominio, servicios, controladores) que `docs/mediciones/jacoco/REPORTE-JACOCCO.md` no provee todavía de forma separada — pendiente para el cierre. Además, `pom.xml` configura JaCoCo solo para reportar, sin `jacoco:check` que falle el build bajo el umbral (Capítulo 6, brecha declarada también en `docs/observaciones/INFORME-BRECHAS-ENTREGA-FINAL.md`).

8.5. Calidad web (Lighthouse) — RQ1

Métrica: puntajes Performance, Accessibility, Best Practices, SEO (0–100), 3 corridas por perfil (mobile, desktop). **Datos crudos:** `docs/mediciones/lighthouse/lhci-20260817-*-{mobile,desktop}-run{1,2,3}.json`.

Cuadro 8.5: Resultados Lighthouse por perfil y categoría

Categoría	Mobile (3 corridas)	Desktop (3 corridas)	Umbral	Cumple
Performance	81 / 81 / 80	100 / 100 / 100	≥ 80	✓ ambos
Accessibility	93 / 93 / 93	93 / 93 / 93	≥ 90	✓ ambos
Best Practices	96 / 96 / 96	96 / 96 / 96	≥ 90	✓ ambos
SEO	100 / 100 / 100	100 / 100 / 100	≥ 90	✓ ambos

Interpretación: las cuatro categorías cumplen umbral en ambos perfiles, con consistencia prácticamente perfecta entre las 3 corridas de cada perfil (varianza ≤ 1 punto en Performance mobile, 0 en el resto). Los hallazgos reales que impiden el 100/100 están documentados sin maquillaje en `docs/mediciones/lighthouse/REPORTE-LIGHTHOUSE.md`:

`target-size` (elementos táctiles bajo el tamaño mínimo recomendado, Accessibility 93) y un 403 de red esperado en `/api/auth/refresh` al cargar sin sesión (Best Practices 96) — ninguno de los dos es un defecto real de la aplicación, sino comportamiento esperado bajo las condiciones de la auditoría.

8.6. Resumen de reproducibilidad de las figuras y tablas de este capítulo

Todo dato reportado en este capítulo proviene de un archivo crudo versionado bajo `docs/mediciones/` y de un script o comando documentado (ver Capítulo 5 §5.5 y `docs/mediciones/DATA-PROVENANCE.md` para el commit/script exacto de cada medición) — ningún número de esta sección se calculó fuera del repositorio.

9. Discusión

9.1. Respuesta a las preguntas de investigación

RQ1 — ¿Cumple el sistema los umbrales de rendimiento, seguridad, usabilidad, cobertura y calidad web declarados? Parcialmente. De los cinco bloques evaluados (Capítulo 8), cuatro cumplen sus umbrales por completo (rendimiento, seguridad manual+automatizada, usabilidad, calidad web en ambos perfiles). El bloque de cobertura cumple en líneas ($72.0\% \geq 70\%$) pero **no en ramas** ($62.5\% < 70\%$) — una respuesta honesta a RQ1 no puede ser “sí” sin matiz. El sistema cumple la gran mayoría de sus propiedades de calidad declaradas con evidencia reproducible, con una brecha cuantificada y no oculta en cobertura de ramas.

RQ2 — ¿Es la estrategia híbrida de acceso a datos implementable y verificable dentro de las restricciones del proyecto? Sí, parcialmente y con matices declarados. Se conectaron 7 procedimientos/funciones end-to-end al código en ejecución (por encima del mínimo de 6 exigido por la guía), sin concatenación de SQL dinámico detectada por análisis estático ni por auditoría manual (Capítulo 8 §8.2). Sin embargo, las 6 rutinas originales de la Tercera Entrega siguen sin conectar (Capítulo 6, ADR-006), y existe un nuanche de conformidad literal sobre el mecanismo de invocación (`@Query(nativeQuery=true)`) parametrizado en vez de `@Procedure/@NamedStoredProcedureQuery` estrictos, Capítulo 7 §7.4) que el equipo debe resolver antes del cierre.

RQ3 — ¿Produce el proceso de ingeniería de requisitos aplicado un corpus verificable y trazable? Sí, con una brecha de conformidad sintáctica declarada. El 100 % de los 37 requisitos tiene trazabilidad completa hacia código y pruebas (`matriz.csv`, validado en CI), y el 89.2 % está implementado o verificado. Nueve de las quince características de calidad INCOSE v4 se cumplen plenamente, pero el corpus no sigue el patrón sintáctico formal de ISO/IEC/IEEE 29148:2018 (característica C9), y dos requisitos violan singularidad (C5) — hallazgos concretos, no una declaración genérica de “cumple parcialmente” (Capítulo 4 §4.3).

RQ4 — ¿Qué amenazas a la validez afectan la generalización de los resultados? Ver Capítulo 10 para el análisis completo por categoría; en resumen, la amenaza más relevante es de validez interna en la comparación caché frío/caliente (diseño experimental que no aísla la variable de interés) y de validez externa en el muestreo por conveniencia del bloque de usabilidad.

9.2. Comparación con trabajos relacionados

Dado que el Capítulo 3 declara honestamente la ausencia de una revisión sistemática de literatura completa, esta comparación se limita a lo que el capítulo sí pudo establecer: ninguna de las referencias contextuales citadas —incluidas las plataformas multilaterales [11], los protocolos de *escrow* verificable [10] y los estudios de microservicios en producción [29]— combina patrón *escrow*, verificación de identidad asistida por IA y estrategia híbrida de acceso a datos evaluada empíricamente en un mismo sistema (§3.3). Una comparación cuantitativa rigurosa (tabla comparativa de ≥ 8 filas con métricas equivalentes) requiere primero completar la revisión de literatura pendiente — no se fabrica aquí una comparación que la evidencia disponible no sostiene.

9.3. Hallazgos inesperados

El hallazgo más relevante no anticipado al inicio del proyecto fue el propio del bloque de rendimiento (§8.1): el escenario “frío” resultó con mejor latencia que el “caliente”, revelando un defecto de diseño del script de carga (no del sistema evaluado) que invalida la comparación tal como está construida. Detectarlo y documentarlo con honestidad, en vez de descartarlo o presentarlo como validado, es en sí mismo parte del aporte metodológico de este trabajo.

9.4. Implicaciones prácticas

Para equipos que evalúen adoptar una estrategia híbrida de acceso a datos similar: la evidencia de este proyecto sugiere que conectar procedimientos almacenados end-to-end (no solo declararlos en SQL) consume un esfuerzo de verificación no trivial —la brecha entre “el archivo `.sql` existe” y “el código Java realmente lo invoca” fue precisamente el hallazgo que motivó la ampliación de ADR-006 documentada en el Capítulo 6— y que declarar la nuanza de conformidad del mecanismo de invocación (Capítulo 7 §7.4) antes de una auditoría externa evita sorpresas en la evaluación.

10. Amenazas a la validez

10.1. Validez de constructo

¿Miden los instrumentos elegidos lo que pretenden medir? El instrumento SUS mide usabilidad percibida, no usabilidad objetiva (tiempo de tarea, tasa de error) — el proyecto no midió usabilidad objetiva como complemento, por lo que la conclusión de “usabilidad aceptable” se apoya en un único constructo subjetivo. **Mitigación aplicada:** se usó el instrumento SUS sin modificar (10 ítems originales de Brooke [5]), evitando la amenaza de validez de constructo que introduciría una versión ad-hoc del cuestionario.

10.2. Validez interna

La comparación caché frío/caliente del bloque de rendimiento (§8.1, §9.1) tiene una amenaza de validez interna real y ya documentada: el diseño del script de carga no varía los parámetros de consulta entre iteraciones, por lo que la variable independiente (estado del caché) no se manipula de forma efectiva durante la mayoría de las 1500 iteraciones por corrida. **Mitigación aplicada:** declarar la limitación explícitamente en vez de presentar la comparación como válida (`docs/mediciones/perf/REPORTE-PERF.md`); **mitigación pendiente:** rediseñar el script para variar parámetros de forma controlada y forzar *cache misses* reales y medibles.

10.3. Validez externa

El bloque de usabilidad usa una muestra de conveniencia (N=16, red de contactos del equipo, ver Capítulo 5 §5.4), no una muestra aleatoria de la población objetivo real (creadores y clientes de economía creativa). Siguiendo la orientación de Baltes y Ralph [34] sobre reporte de muestreo en ingeniería de software empírica, esta limitación se declara explícitamente en vez de generalizar el resultado SUS a “los usuarios de Artisync” sin matiz: el resultado (76.88, categoría B de Bangor [6]) es válido para la muestra evaluada, con generalización externa limitada a un perfil demográfico similar (adultos con experiencia web media-alta, dispositivos variados).

10.4. Validez de conclusión

Las conclusiones de los bloques de rendimiento y cobertura se apoyan en estadística descriptiva (media, DT, IC 95 %) sin test inferencial, porque no se identificaron comparaciones de dos condiciones estadísticamente válidas para probar dentro del alcance actual (la única candidata, caché frío/caliente, está invalidada por la amenaza de validez interna de §10.2). Esto es consistente con la recomendación de preferir IC 95 % sobre valores p como reporte primario (Capítulo 5 §5.6), siguiendo el protocolo estadístico de Arcuri y Briand [36] para ingeniería de software, pero implica que ninguna afirmación de “diferencia significativa” se sostiene en este documento — y en efecto, ninguna se hace.

11. Trabajo futuro

1. **Conectar las 6 rutinas SQL originales de la Tercera Entrega** (`fn_catalogo_filtrado`, `fn_calificacion_promedio_creador`, `fn_reporte_comisiones_creador`, `fn_cerrar_pedidos_vencidos`, `fn_liberar_fondos_escrow`, `fn_generar_codigo_pedido`) al código Java en ejecución, cerrando la deuda técnica admitida en ADR-006. Quedó fuera del alcance de la ampliación del 16 de agosto porque esa ampliación priorizó sumar rutinas nuevas del módulo de seguridad, no reparar las existentes.
2. **Rediseñar el script de carga k6** para forzar *cache misses* reales y medibles por iteración, habilitando un test inferencial (Wilcoxon) con tamaño de efecto (*Cliff's delta*) válido sobre la comparación caché frío/caliente [36] — hoy inválida por la amenaza de validez interna documentada en §10.2.
3. **Elevar la cobertura de ramas de JaCoCo** del 62.5 % actual al umbral de 70 % exigido, con desglose por módulo (dominio/servicio/controlador) en vez del agregado global actual, y activar `jacoco:check` en `pom.xml` para que el build falle bajo el umbral en vez de solo reportarlo.
4. **Ejecutar una revisión de literatura reducida** (2–3 bases de datos, 10–15 términos clave) para completar el Capítulo 3 con la tabla comparativa de ≥ 8 filas que la guía exige y que este documento declaró honestamente como pendiente.
5. **Resolver la nuance de conformidad de invocación de procedimientos** (Capítulo 7 §7.4): decidir entre reescribir las 7 funciones nuevas como `PROCEDURE` para usar `@Procedure` en sentido estricto, adoptar `@NamedStoredProcedureQuery`, o documentar formalmente por qué `@Query(nativeQuery=true)` parametrizado satisface el requisito.

12. Conclusiones

Este trabajo diseñó, implementó y evaluó empíricamente Artisync, una plataforma web de intermediación entre creadores de contenido digital y clientes, con una estrategia híbrida de acceso a datos y verificación de identidad asistida por inteligencia artificial. Respondiendo brevemente a las preguntas de investigación planteadas en el Capítulo 1: el sistema cumple la mayoría de sus umbrales de calidad declarados con evidencia empírica reproducible (RQ1), con una brecha cuantificada en cobertura de ramas; la estrategia híbrida de acceso a datos es implementable y verificable dentro de las restricciones de un proyecto académico de 17 semanas, con 7 procedimientos conectados end-to-end y una nuanza de conformidad declarada sobre el mecanismo de invocación (RQ2); el proceso de ingeniería de requisitos produjo un corpus 100 % trazable hacia código y pruebas, con brechas concretas de conformidad sintáctica INCOSE v4 (RQ3); y las amenazas a la validez más relevantes —diseño del experimento de caché y muestreo de usabilidad por conveniencia— están documentadas y mitigadas en la medida de lo posible dentro del alcance del proyecto (RQ4).

Las cinco contribuciones enumeradas en el Capítulo 1 §1.4 se sostienen con evidencia concreta de los Capítulos 6 a 8: el sistema software completo y desplegable (Capítulos 6–7), la estrategia híbrida de acceso a datos verificada end-to-end (Capítulo 7 §7.4), el corpus de ingeniería de requisitos trazable (Capítulo 4), el paquete de evidencia empírica reproducible (Capítulo 8, con procedencia declarada en `docs/mediciones/DATA-PROVENANCE.md`), y los siete ADR (Capítulo 6 §6.2).

El valor distintivo de este documento, más allá de reportar únicamente resultados favorables, es la disciplina de declarar honestamente cada brecha encontrada durante su propia redacción —desde la cobertura de ramas por debajo del umbral hasta la nuanza de conformidad de invocación de procedimientos— siguiendo el mismo estándar de auditoría verificable que produjo `docs/observaciones/INFORME-BRECHAS-ENTREGA-FINAL.md` y que este documento hereda explícitamente.

13. Declaraciones obligatorias

13.1. Disponibilidad de código

Repositorio público: <https://github.com/JohanCarvajal04/Proyecto-WEB-ARTISYNC>. Etiqueta de esta entrega: `v1.0.0`, commit `d07656b` (el commit archivado por el release de GitHub y por el DOI de Zenodo). El tag `v1.0.0` existe también en la copia local del repositorio apuntando a un commit distinto (`5b61f86`), creado por separado; el equipo debe unificar ambos tags antes del cierre para que `git describe` no sea ambiguo. DOI Zenodo del software: `10.5281/zenodo.21978572` (archivo de `v1.0.0`; la versión anterior, `v0.9.0-rc`, quedó archivada aparte en `10.5281/zenodo.21730559`).

13.2. Disponibilidad de datos

Depósito Zenodo separado del software para el dataset de mediciones (`docs/mediciones/`), con DOI propio y licencia CC BY 4.0, siguiendo el principio de citación independiente de software y datos. Brecha declarada en `docs/observaciones/INFORME-BRECHAS-ENTREGA-FINAL.md`, Bloque D.3 y en `docs/checklists/fair-checklist.md`, ítem Findable #1. Mientras tanto, los datos crudos están disponibles en el mismo repositorio bajo `docs/mediciones/`, con licencia MIT del repositorio (no la licencia de datos recomendada — ver la misma brecha).

13.3. Disponibilidad de materiales

- Colección Postman (26 peticiones): `Pruebas.postman_collection.json`.
- Script de carga k6: `k6/catalogo-load.js`.
- Configuración Lighthouse: `artisync/Frontend/lighthousec.mobile.json`, `lighthousec.desktop.json`.
- Scripts de análisis SUS: `docs/mediciones/sus/analisis-sus.py`, `graficar-sus.py`.
- Protocolo SUS y plantilla de consentimiento: `docs/mediciones/sus/instrucciones-formulario.md`, `docs/etica/consentimientos/plantilla.md`.

13.4. Declaración ética

Ver `docs/etica/ETHICS.md` para el detalle completo: resguardo de consentimientos informados fuera del repositorio público, participación voluntaria sin compensación, anonimización por código de participante (P01–P16), sin grabación de audio/video.

13.5. Contribuciones de autores (taxonomía CRediT)

Ver la matriz completa de los catorce roles CRediT en `CONTRIBUTORS.md`. Resumen: los cuatro integrantes (Bone Arroyo N., Carvajal Loor J., Figueroa Morales B., Rios Cuyabazo J.) comparten Conceptualización, Metodología, Software, Validación, Investigación, Redacción (borrador y revisión) y Administración del proyecto; Análisis formal se concentra en Figueroa B. y Rios J.; Gestión de datos en Rios J.; Visualización en Bone N. y Carvajal J. El propio `CONTRIBUTORS.md` deja abierto un ajuste final de precisión antes del cierre — pendiente de confirmación del equipo.

13.6. Declaración de conflictos de interés

Ver `docs/etica/ETHICS.md`: el equipo declara ausencia de conflictos de interés.

13.7. Declaración de financiamiento

Ver `docs/etica/ETHICS.md`: sin financiamiento externo; desarrollado con recursos propios del equipo y niveles gratuitos/académicos de proveedores cloud.

13.8. Declaración de uso de asistentes de inteligencia artificial

Ver `docs/etica/ai-disclosure.md` para el detalle completo (herramienta: Claude Code / Claude Sonnet 5; fases del proyecto donde se usó; qué no se generó con IA; nota de honestidad sobre el alcance de la declaración). Esta versión del documento (conversión a LaTeX, verificación bibliográfica contra fuente primaria y ampliación de referencias) se produjo con la misma herramienta y bajo el mismo alcance declarado — El equipo debe confirmar que `ai-disclosure.md` referencia explícitamente esta sesión de trabajo (17-08-2026) antes del cierre.

13.9. Agradecimientos

Agradecemos sinceramente a todas las personas e instituciones que han hecho posible el desarrollo de este proyecto. Esta experiencia ha sido invaluable para nuestro crecimiento tanto profesional como personal, brindándonos la oportunidad de profundizar nuestros conocimientos prácticos y aprender más sobre el desarrollo de software, las arquitecturas escalables y el diseño centrado en el usuario.

Queremos extender un profundo agradecimiento a todos los usuarios que, de forma anónima y desinteresada, participaron en nuestras pruebas de usabilidad (evaluación SUS). Su tiempo, disposición y valiosa retroalimentación colectiva fueron fundamentales para iterar y mejorar significativamente la experiencia de la plataforma Artisync.

Finalmente, agradecemos a nuestros docentes y a la institución por la formación, guía y apoyo constante a lo largo de todo este proceso de aprendizaje.

Bibliografía

- [1] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.
- [2] “ISO/IEC/IEEE 29148:2018: Systems and software engineering — life cycle processes — requirements engineering,” tech. rep., International Organization for Standardization / IEC / IEEE, 2018.
- [3] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Heidelberg: Springer, 2010.
- [4] OWASP Foundation, “OWASP top 10:2021 — the ten most critical web application security risks,” 2021.
- [5] J. Brooke, “SUS: A “quick and dirty” usability scale,” in *Usability Evaluation in Industry* (P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, eds.), pp. 189–194, London: Taylor & Francis, 1996.
- [6] A. Bangor, P. Kortum, and J. Miller, “Determining what individual SUS scores mean: Adding an adjective rating scale,” *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.
- [7] “ISO/IEC 25010:2011: Systems and software engineering — systems and software quality requirements and evaluation (square) — system and software quality models,” tech. rep., International Organization for Standardization, 2011.
- [8] R. Peres, M. Schreier, D. A. Schweidel, and A. Sorescu, “The creator economy: An introduction and a call for scholarly research,” *International Journal of Research in Marketing*, vol. 41, no. 3, pp. 403–410, 2024.
- [9] L. Gussek, A. Grabbe, and M. Wiesche, “Challenges of IT freelancers on digital labor platforms: A topic model approach,” *Electronic Markets*, vol. 33, no. 1, p. 55, 2023.
- [10] A. Asgaonkar and B. Krishnamachari, “Solving the buyer and seller’s dilemma: A dual-deposit escrow smart contract for provably cheat-proof delivery and payment for a digital good without a trusted mediator,” in *2019 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pp. 262–267, 2019.

- [11] A. Hagiú and J. Wright, “Multi-sided platforms,” *International Journal of Industrial Organization*, vol. 43, pp. 162–174, 2015.
- [12] International Council on Systems Engineering (INCOSE), “Guide to writing requirements,” INCOSE Technical Product INCOSE-TP-2010-006-04, INCOSE, July 2023.
- [13] H. Zhu, P. A. V. Hall, and J. H. R. May, “Software unit test coverage and adequacy,” *ACM Computing Surveys*, vol. 29, no. 4, pp. 366–427, 1997.
- [14] L. Inozemtseva and R. Holmes, “Coverage is not strongly correlated with test suite effectiveness,” in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pp. 435–445, 2014.
- [15] OWASP Foundation, “SQL injection prevention cheat sheet.” OWASP Cheat Sheet Series.
- [16] W. G. J. Halfond and A. Orso, “AMNESIA: Analysis and monitoring for NEutralizing SQL-injection Attacks,” in *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 174–183, 2005.
- [17] C. Gould, Z. Su, and P. Devanbu, “Static checking of dynamically generated queries in database applications,” in *Proceedings of the 26th International Conference on Software Engineering (ICSE)*, pp. 645–654, 2004.
- [18] A. Cockburn, *Writing Effective Use Cases*. Addison-Wesley, 2000.
- [19] M. Nygard, “Documenting architecture decisions.” Cognitect Blog, Nov. 2011.
- [20] K. Wiegers and J. Beatty, *Software Requirements*. Redmond, WA: Microsoft Press, 3rd ed., 2013.
- [21] S. Brown, “The C4 model for software architecture.” InfoQ, June 2018. Ver también <https://c4model.com>.
- [22] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [23] M. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015.
- [24] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” EBSE Technical Report EBSE-2007-01, EBSE (Keele University / University of Durham), 2007.

- [25] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher, “The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021.
- [26] S. Coello Quezada, O. Peñafiel Chele, and Á. Yanza Montalván, “Aplicación web e-commerce B2C para la ferretería “el descanso” mediante tecnologías de desarrollo front-end y back-end,” *Investigación, Tecnología e Innovación*, vol. 15, no. 19, 2023. Universidad de Guayaquil.
- [27] P. Monsalve-Obreque, P. Vargas-Villaruel, Y. Hormazábal-Astorga, J. Hochstetter-Diez, J. Bustos-Gómez, and M. Diéguez-Rebolledo, “Proposal to improve the E-Commerce platform development process with an exploratory case study in chile,” *Applied Sciences*, vol. 13, no. 14, p. 8362, 2023.
- [28] T. Wulfert and R. Schütte, “Retailer’s dual role in digital marketplaces: Reference architectures for retail information systems,” *SN Computer Science*, vol. 3, no. 3, p. 206, 2022.
- [29] Y. Wang, H. Kadiyala, and J. Rubin, “Promises and challenges of microservices: an exploratory study,” *Empirical Software Engineering*, vol. 26, p. 63, 2021.
- [30] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [31] P. Ralph, N. bin Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri, B. B. N. de França, C. A. Furia, G. Gay, N. Gold, D. Graziotin, P. He, R. Hoda, N. Juristo, B. Kitchenham, V. Lenarduzzi, J. Martínez, J. Melegati, D. Mendez, T. Menzies, J. Moller, D. Pfahl, R. Robbes, D. Russo, N. Saarimäki, F. Sarro, J. Siegmund, D. Spinellis, M. Staron, K.-J. Stol, M.-A. Storey, D. Taibi, D. Tamburri, M. Torchiano, C. Treude, B. Turhan, S. Vegas, and X. Wang, “Empirical standards for software engineering research,” *ACM SIGSOFT Software Engineering Notes*, vol. 46, no. 1, p. 19, 2021. Versión extendida en <https://arxiv.org/abs/2010.03525>.
- [32] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.

- [33] V. R. Basili, G. Caldiera, and H. D. Rombach, “The goal question metric approach,” in *Encyclopedia of Software Engineering*, vol. 1, pp. 528–532, John Wiley & Sons, 1994.
- [34] S. Baltes and P. Ralph, “Sampling in software engineering research: a critical review and guidelines,” *Empirical Software Engineering*, vol. 27, no. 4, p. 94, 2022.
- [35] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [36] A. Arcuri and L. Briand, “A practical guide for using statistical tests to assess randomized algorithms in software engineering,” in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pp. 1–10, 2011.
- [37] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, *et al.*, “The FAIR guiding principles for scientific data management and stewardship,” *Scientific Data*, vol. 3, p. 160018, 2016.

A. Tabla de observaciones acumuladas

Ver docs/observaciones/OBSERVACIONES.md (25 observaciones, 68 % resueltas, 8 % parcialmente resueltas, 24 % pendientes, con razones declaradas).

B. Checklist Ralph et al. 2021

Ver docs/checklists/ralph-2021-checklist.md [31].

C. Checklist FAIR

Ver docs/checklists/fair-checklist.md [37].

D. Checklist INCOSE v4

Ver docs/checklists/incose-checklist.md [12].

E. Checklist PRISMA 2020

Ver `docs/checklists/prisma-2020-checklist.md` [25] (resuelto como “No Aplica”, justificado — ver Capítulo 3 §3.1).

F. Matriz de trazabilidad completa

Ver docs/trazabilidad/matriz.csv.

G. Protocolo SUS completo (10 preguntas + escala)

Ver docs/mediciones/sus/instrucciones-formulario.md [5].

H. Capturas de ejecución del pipeline CI

El equipo debe adjuntar capturas de pantalla de 3 ejecuciones consecutivas exitosas del workflow `.github/workflows/ci.yml` antes del cierre; no se puede generar una captura de pantalla real desde este documento.

I. Cadena de búsqueda completa del

Capítulo 3

Depende de ejecutar la revisión de literatura reducida recomendada en el Capítulo 3 §3.1. Las búsquedas puntuales ejecutadas para verificar y ampliar la bibliografía de esta versión (Anexo J) no siguen una cadena de búsqueda reproducible sobre bases de datos bibliográficas y no sustituyen este anexo.

J. Clasificación de impacto de la bibliografía

Este anexo es nuevo en esta versión del documento. Clasifica cada referencia de `referencias.bib` según el criterio de la guía de la Entrega Final (“alto impacto” = indexada Scopus/JCR Q1–Q2, o publicada en las actas de ICSE, FSE, ASE, MSR, EASE o ESEM), con la fuente de la clasificación. Ninguna referencia se clasificó como “alto impacto” sin verificar su `venue/journal` contra una fuente externa (ver `README.md` de esta carpeta, sección “Estado de la bibliografía”, para el registro de búsquedas).

Cuadro J.1: Clasificación de impacto por referencia

Clave	Venue	Alto im- pacto	Motivo
HALFOND2005	ASE 2005	Sí	Venue listado
GOULD2004	ICSE 2004	Sí	Venue listado
INOZEMTSEVA2005	ICSE 2014	Sí	Venue listado
ARCURI2011	ICSE 2011	Sí	Venue listado
WANG2021	Empirical Software Engineering 26	Sí	JCR Q1 (SE)
RUNESON2009	Empirical Software Engineering 14(2)	Sí	JCR Q1 (SE)
BALTES2022	Empirical Software Engineering 27(4)	Sí	JCR Q1 (SE)
ZHU1997	ACM Computing Surveys 29(4)	Sí	JCR Q1
PRISMA2021	BMJ 372	Sí	JCR Q1
WILKINSON2016	Scientific Data 3	Sí	JCR Q1
HEVNER2004	MIS Quarterly 28(1)	Sí	JCR Q1 (SI)
PEFFERS2007	J. of Management Information Systems 24(3)	Sí	JCR Q1/Q2 (SI)
HAGIU2015	Int. J. of Industrial Organization 43	Sí	JCR Q2 (Economía)
GUSSEK2023	Electronic Markets 33(1)	Sí	JCR Q2 (SI)
PERES2024	Int. J. of Research in Marketing 41(3)	Sí	JCR Q1 (Marketing)
MONSALVE2023	Applied Sciences (MDPI) 13(14)	Sí	JCR Q2
KITCHENHAM2007	EBSE Technical Report	No	Informe técnico, no journal/conferencia
RALPH2021	ACM SIGSOFT Software Eng. Notes 46(1)	No	Boletín, no journal JCR
BASILII1994	Encyclopedia of Software Engineering	No	Entrada de encyclopedia
BROOKE1996	Usability Evaluation in Industry (cap. de libro)	No	Capítulo de libro
BANGOR2009	Journal of Usability Studies 4(3)	No	No indexada JCR
WOHLIN2012	Libro (Springer)	No	Libro, no journal/conferencia
WULFERT2022	SN Computer Science 3(3)	No	Sin ranking JCR consolidado a la fecha
COELLO2023	Investigación, Tecnología e Innovación 15(19)	No	Revista regional, no indexada Scopus/JCR
ASGAONKAR2019	IEEE ICBC 2019	No	Venue de blockchain, no listado en la rúbrica
POHL2010	Libros/guías/blog/disertación	No	Fuente primaria

Totales: 37 referencias en `referencias.bib`; 32 corresponden a literatura académica/técnica citable como parte del mínimo de 30 (excluye los 5 estándares/RFC, contados aparte); de esas 32, **16 se clasifican como “alto impacto”** según el criterio estricto de la guía — por debajo del mínimo de 20 exigido. **Esta brecha se declara honestamente**, igual que el resto del documento: no se reclasificaron referencias de forma optimista para alcanzar el número, y no se añadieron referencias de relevancia dudosa solo para acercarse al umbral (ver README.md, “Por qué nos detuvimos en 16”). Cerrar la brecha requiere la revisión de literatura reducida recomendada en el Capítulo 3 §3.1 y en el Anexo I.